

HelpDesk CRM

Project Report

A detailed overview of the HelpDesk CRM application, Web API, database design, API integration and implementation.



CRM Application
(ASP.NET Core MVC)



Web API
(ASP.NET Core)



Database
(SQL Server / MySQL)



API Integration
(External Services)

PUBLIC GITHUB EDITION

This document provides a consolidated overview of the project, including architecture, modules, database design, API documentation, workflows and implementation details.

SECTION 1 — CRM APPLICATION

1.1 Introduction

The Customer Relationship Management (CRM) application was developed as the primary user-facing component of the HelpDesk CRM project during the internship. The application was designed using **ASP.NET Core MVC** and provides a structured interface for managing customers, employees, support tickets, and administrative operations.

The CRM application acts as the central interface through which users can perform day-to-day operations such as creating and managing customer records, maintaining employee information, assigning and tracking support tickets, and accessing administrative functionality.

The development process involved implementing the application incrementally, beginning with the basic MVC structure and database connectivity and subsequently adding functional modules, validation, authentication, API integration, and UI enhancements.

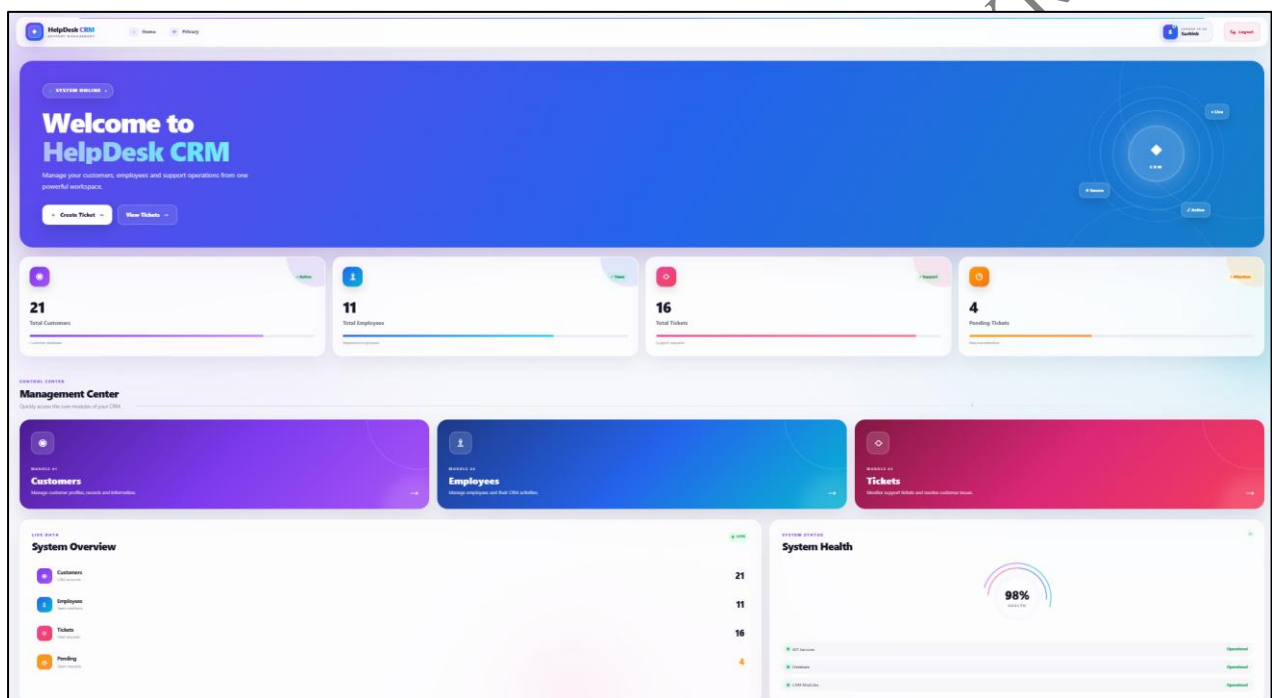


Figure 1.1 — Overall CRM Application Interface

1.2 Objective of the CRM Application

The primary objective of the CRM application was to provide a centralized system for managing customer-support operations.

The major objectives were:

- To maintain customer information in a structured manner.
- To manage employee records and their assigned activities.
- To create and manage support tickets.
- To establish relationships between customers, employees, and tickets.
- To provide administrative authentication and account management.
- To implement client-side validation for user input.
- To provide a dynamic and user-friendly interface.
- To integrate the CRM application with backend APIs.
- To maintain persistent information using a relational database.

1.3 Technology Stack

The CRM application was developed using the following technologies:

Technology	Purpose
ASP.NET Core MVC	Application framework
C#	Backend programming language
Razor Views	Dynamic UI rendering
HTML5	Page structure
CSS3	Styling and layout
JavaScript	Client-side functionality
jQuery	Client-side operations and AJAX
Entity Framework Core	Database interaction
SQL Server / MySQL	Data storage
Visual Studio	Development environment

The application follows the MVC architectural pattern, separating the application's data, business/request-handling logic, and presentation layer.

1.4 CRM Application Architecture

The CRM application follows the **Model–View–Controller (MVC)** architecture.

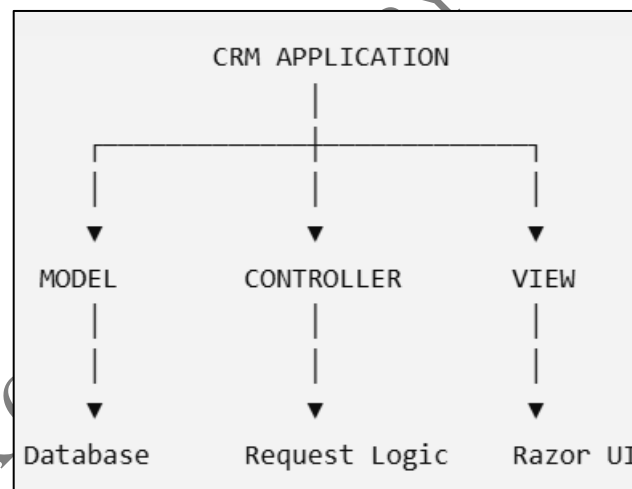


Figure 1.2 — MVC Architecture of the CRM Application

Model

The Model represents the application's data and its structure. Models were created for entities such as customers, employees, tickets, and administrative information.

View

Views provide the user interface of the application. Razor-based.cshtml files were used to display forms, tables, buttons, validation messages, and dynamically generated information.

Controller

Controllers handle incoming requests from the UI, process the required operations, communicate with services/database components, and return the appropriate response or view.

1.5 CRM Project Structure

The application was organized into separate components to maintain a clear and maintainable structure.

A simplified structure of the project is:

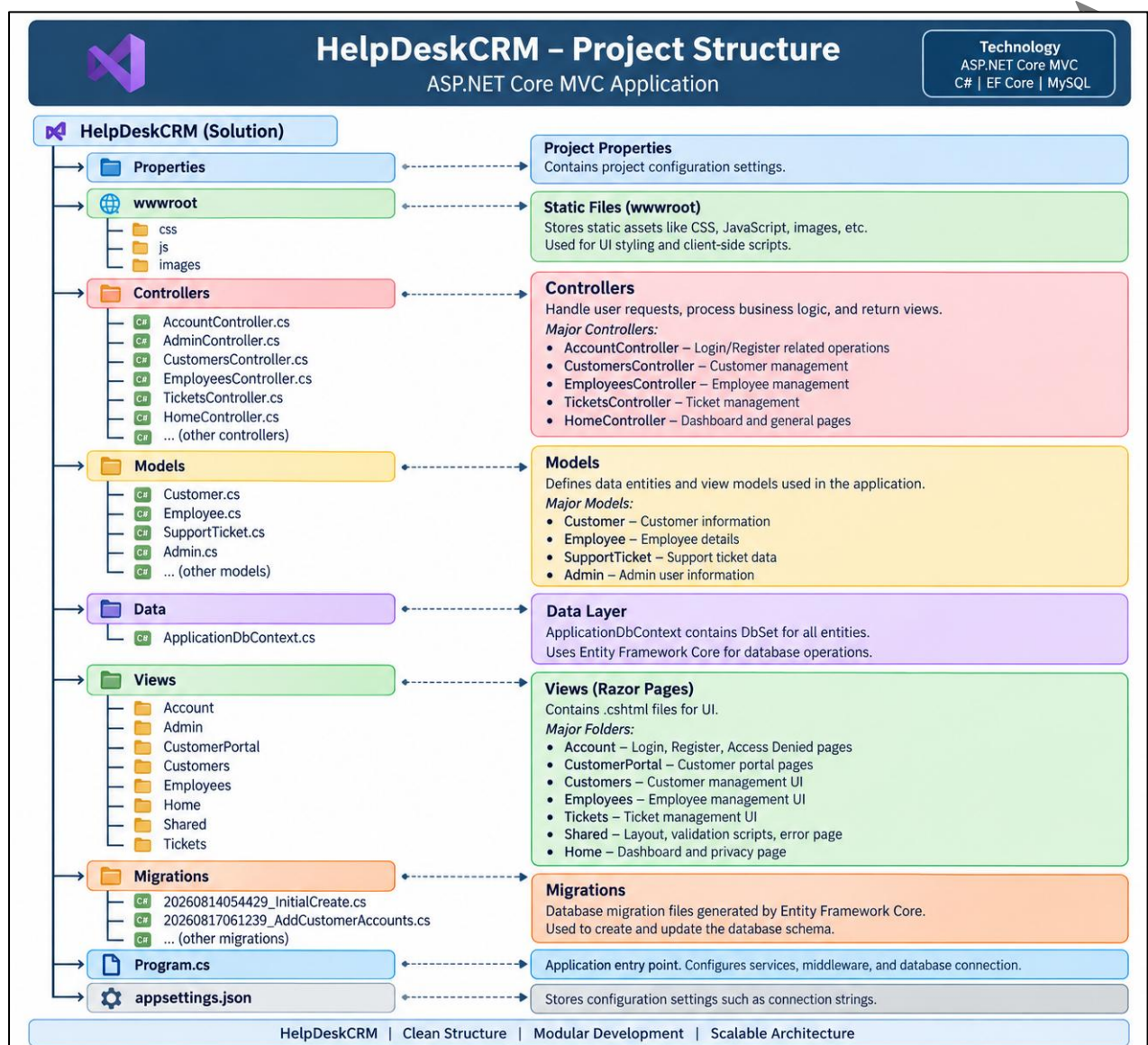


Figure 1.3 — CRM Project Structure in Visual Studio

The separation of responsibilities made it easier to develop, test, debug, and modify individual modules without affecting the complete application.

1.6 Dashboard / Home Page

The Home page was designed as the main entry point of the CRM application. It provides access to the primary services available within the system.

The dashboard was designed around the major functional areas:

- Customer Management
- Support Tickets
- Employee Management
- Authentication

The interface was progressively enhanced to make it more dynamic, visually organized, and user-friendly.

The Authentication functionality was positioned separately from the main management modules, while the primary CRM operations were presented as easily accessible sections/cards.

1.7 Customer Management

The Customer Management module was developed to maintain customer information within the CRM system.

The module provides functionality for:

- Adding new customers
- Viewing customer information
- Searching for customers
- Updating customer information
- Deleting customer records
- Maintaining customer IDs
- Associating customers with support tickets
- Validating customer input

A structured customer form was implemented so that relevant customer information could be entered and stored in the database.

Customer ID

A specific customer ID format was maintained as part of the project requirements. The application was designed to maintain a **three-digit Customer ID**, providing a consistent identification mechanism for customer records.

Customer Search

The customer management functionality also supports searching for customer information. When customer details are retrieved, relevant ticket information can also be presented, allowing the user to understand the customer's support history.

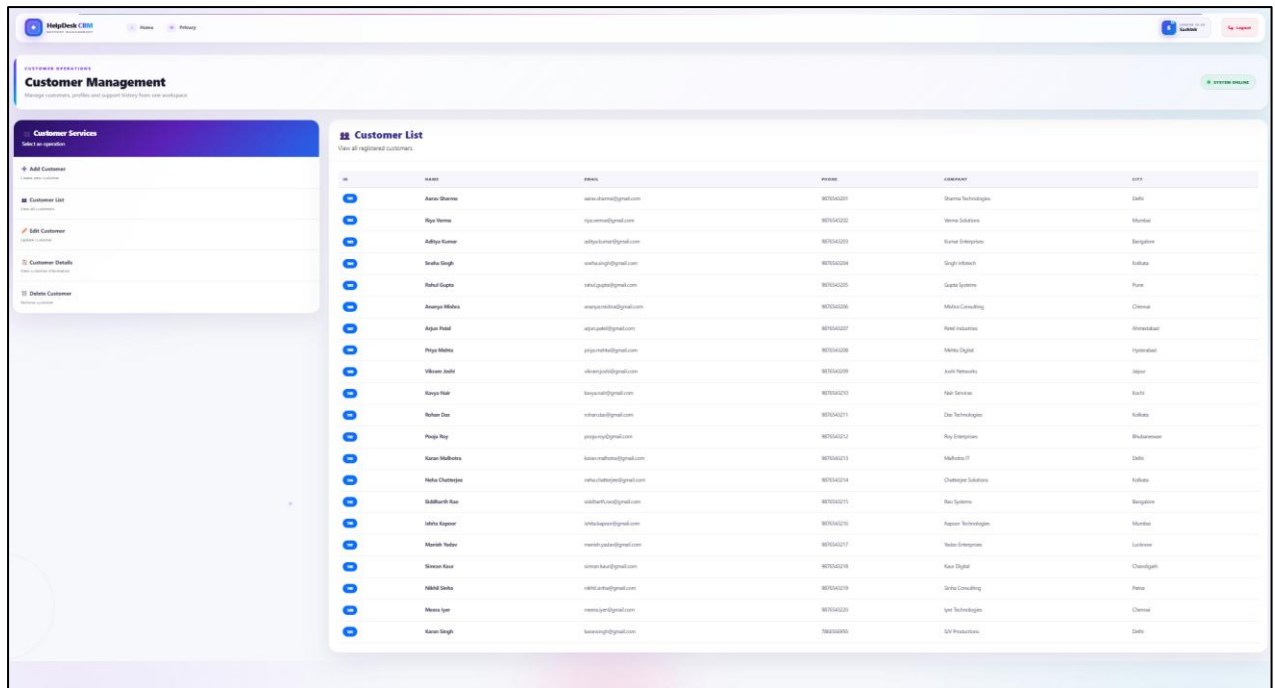


Figure 1.4 — Customer Management Interface

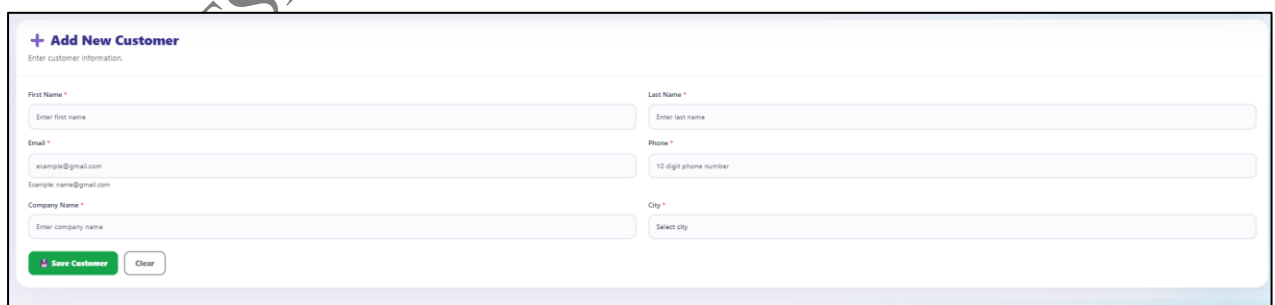
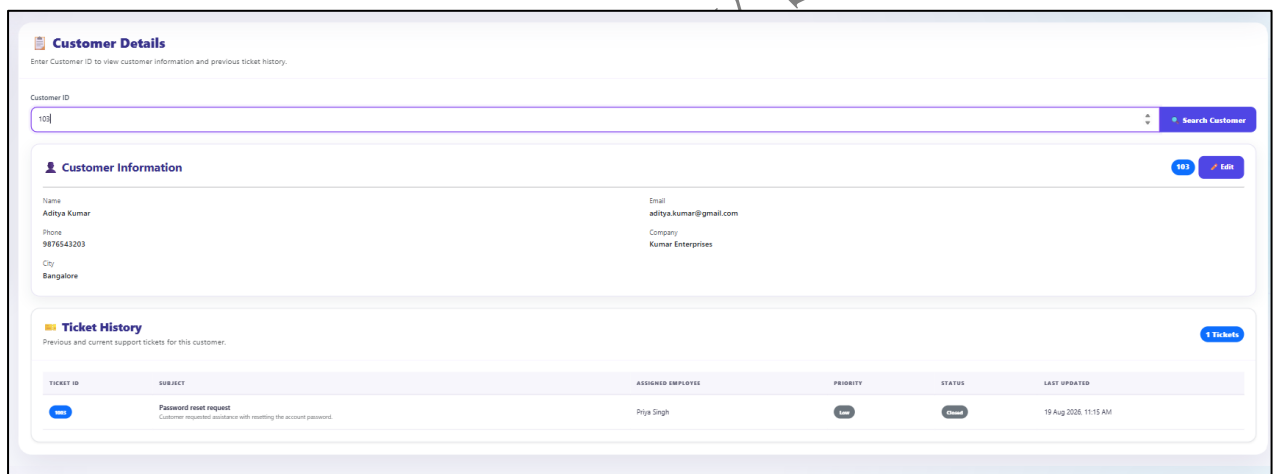


Figure 1.5 — Add/Search Customer Interface

1.8 Employee Management

The Employee Management module was developed to maintain employee information and provide visibility into employee-related activities.

The module includes:

- Employee creation
- Employee information management
- Employee listing
- Employee search/details
- Employee ID management
- Assigned task information
- Employee deletion

A **five-digit Employee ID** format was maintained according to the project requirements.

The employee management interface was also redesigned during development to provide a more modern and interactive experience.

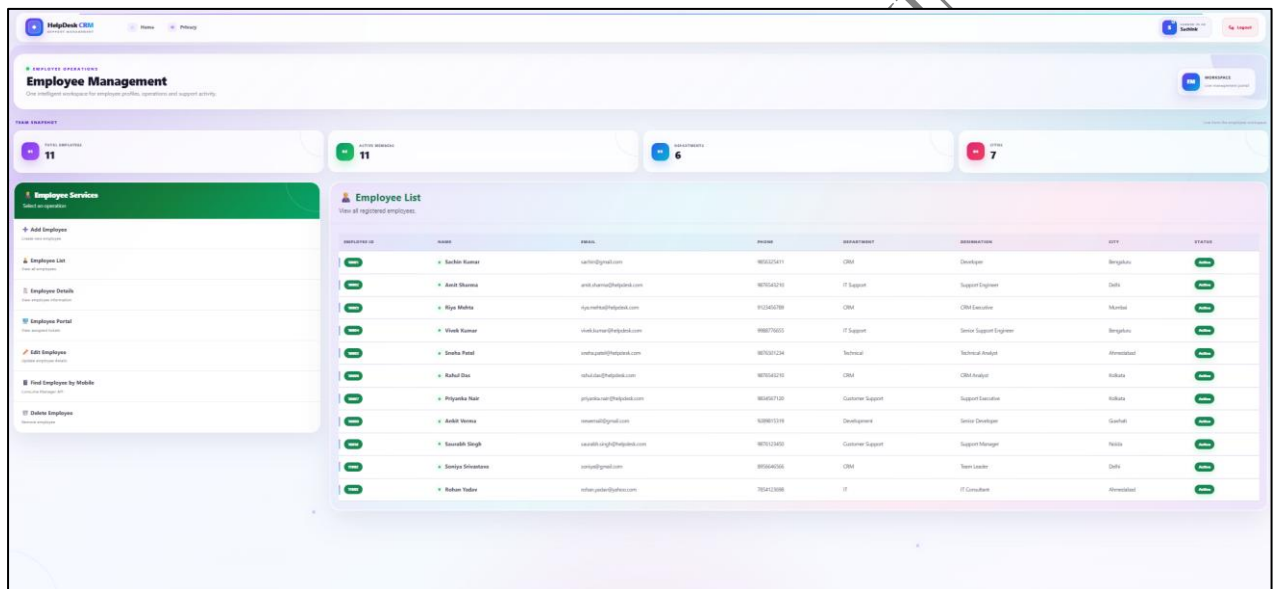


Figure 1.6 — Employee Management Interface

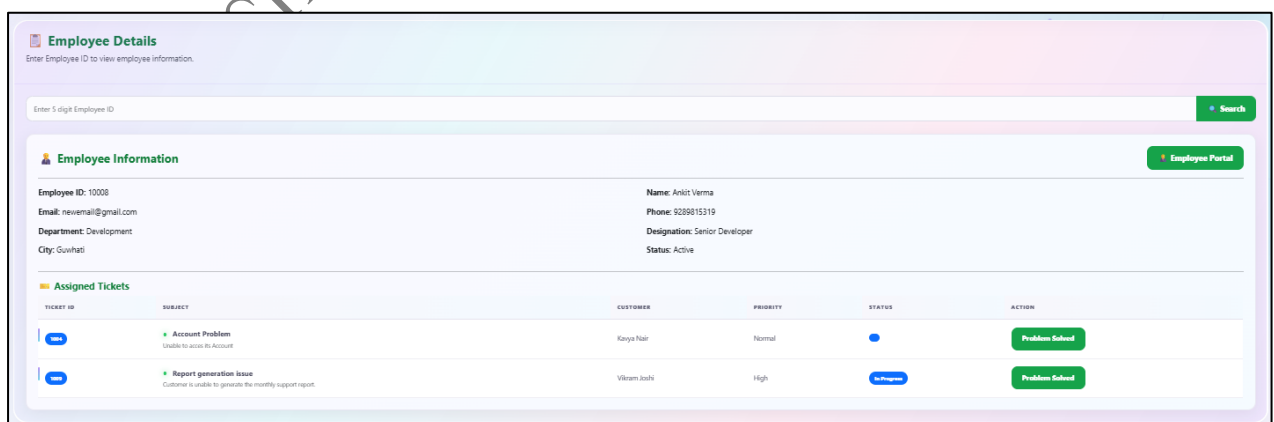


Figure 1.7 — Employee Details and Assigned Tasks

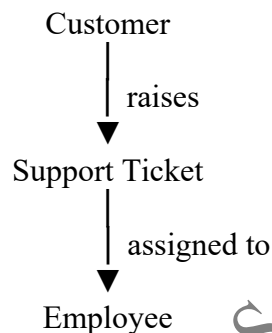
1.9 Ticket Management

The Ticket Management module forms an important part of the HelpDesk CRM because it connects customer support requirements with employees responsible for handling them.

The ticket module was designed to support:

- Ticket creation
- Customer association
- Employee assignment
- Ticket information management
- Ticket status/details
- Ticket updates

The relationship between customers, employees, and tickets allows the application to represent a basic support workflow.



This structure allows support-related activities to be managed from a centralized CRM interface.

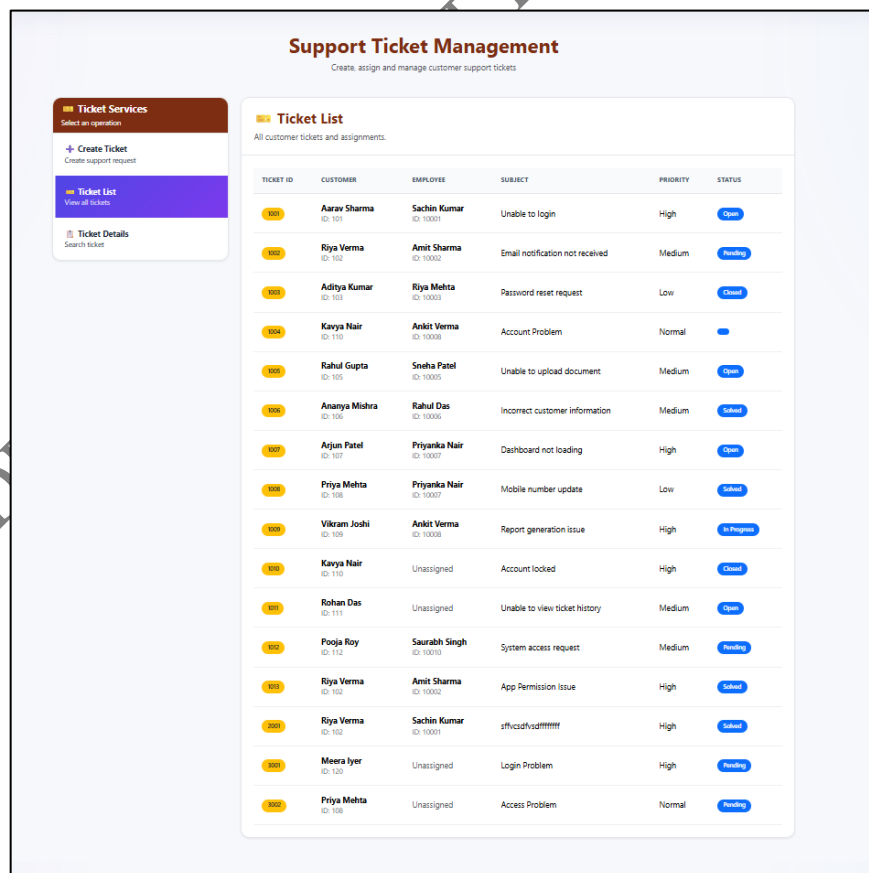


Figure 1.8 — Ticket Management Interface

1.10 Admin Authentication

An administrative authentication module was implemented to control access to administrative functionality.

The authentication section included:

- Admin Login
- Create Admin
- User ID validation
- Duplicate User ID checking
- Login confirmation

The Create Admin functionality was designed to verify whether the requested User ID already exists before creating a new administrator record.

The login workflow was also customized according to the project requirements, including displaying a successful login message without unnecessarily redirecting the user.

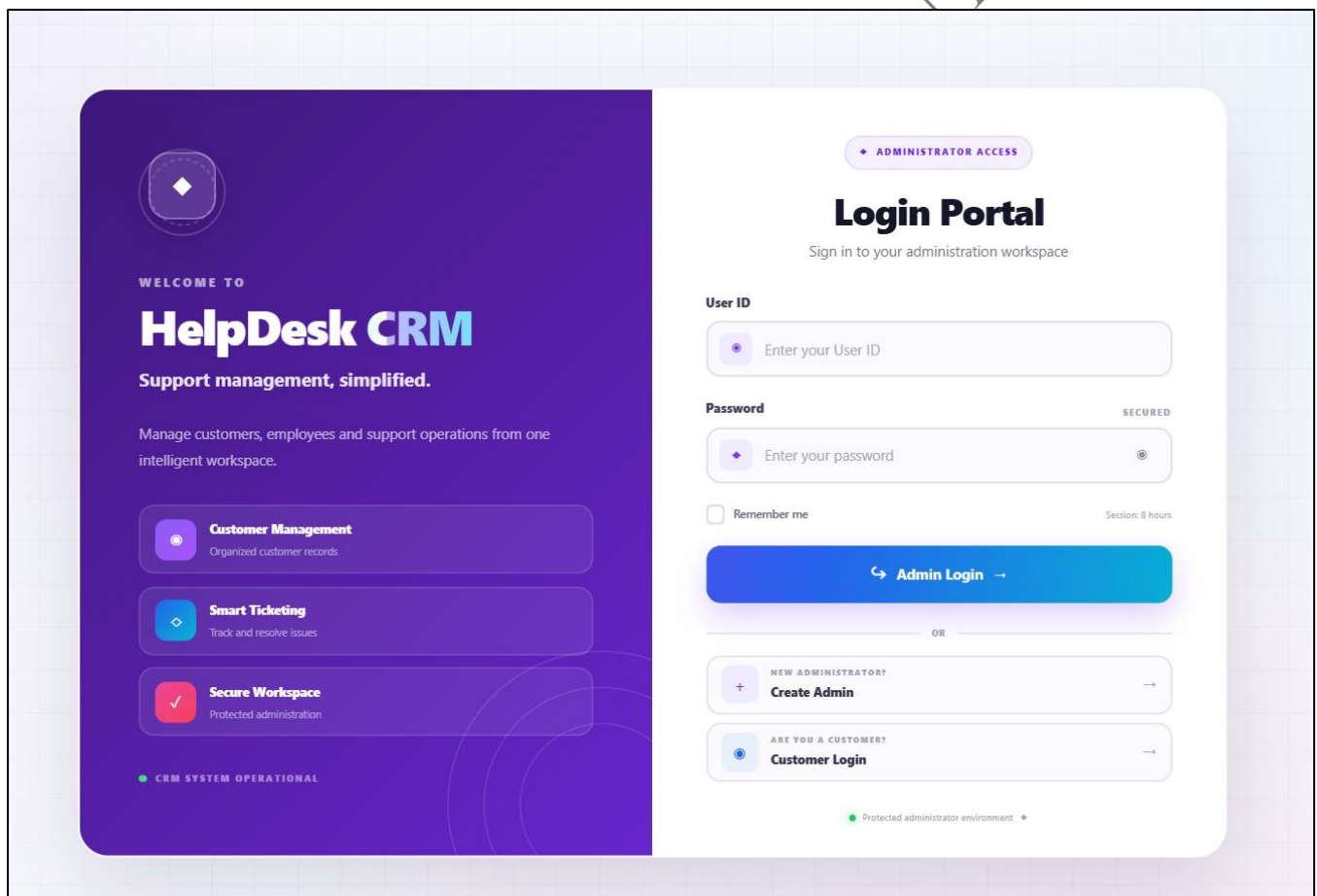


Figure 1.10 — Admin Login Interface

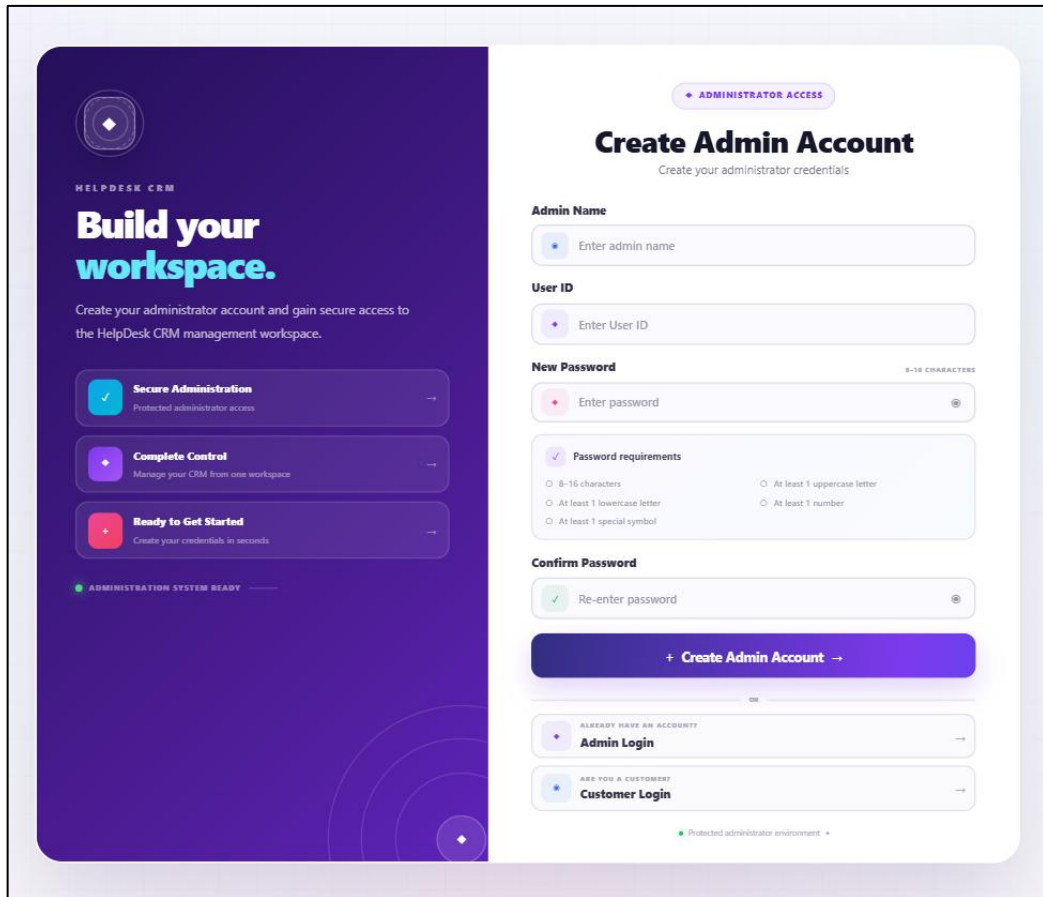


Figure 1.11 — Create Admin Interface

1.11 Client-Side Validation

One of the important improvements made during the CRM development was the implementation of **client-side validation**.

Instead of relying entirely on server-side validation messages, JavaScript/jQuery was used to validate user input directly within the browser.

Validation was implemented for fields such as:

- Required fields
- Email address
- Mobile/phone number
- Input formats
- Other module-specific fields

For example, phone numbers were required to follow the specified **10-digit format**, while email fields were validated against an appropriate email pattern.

A popup-style validation mechanism was also introduced to provide immediate feedback to the user.

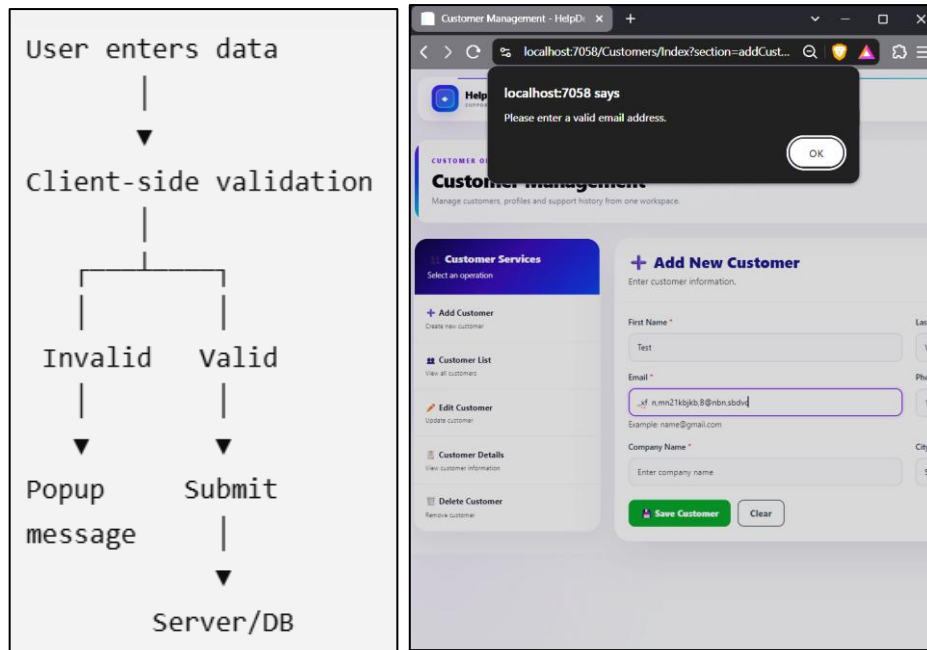


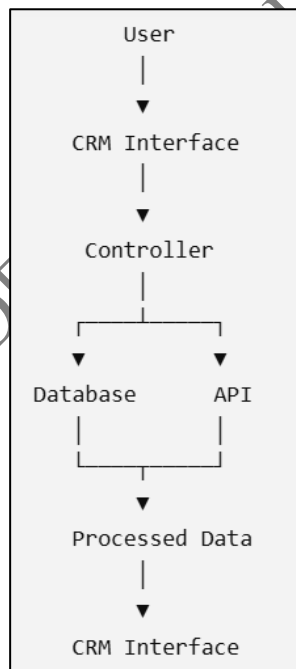
Figure 1.12 — Client-Side Validation Popup

This approach improved the user experience by allowing incorrect inputs to be identified before unnecessary server requests were made.

1.12 Integration with Backend Components

Although the CRM application provides the user interface, it operates as part of a larger system. The application communicates with the database through the application's data-access layer and also interacts with backend APIs for specific functionality.

The overall flow can be represented as:

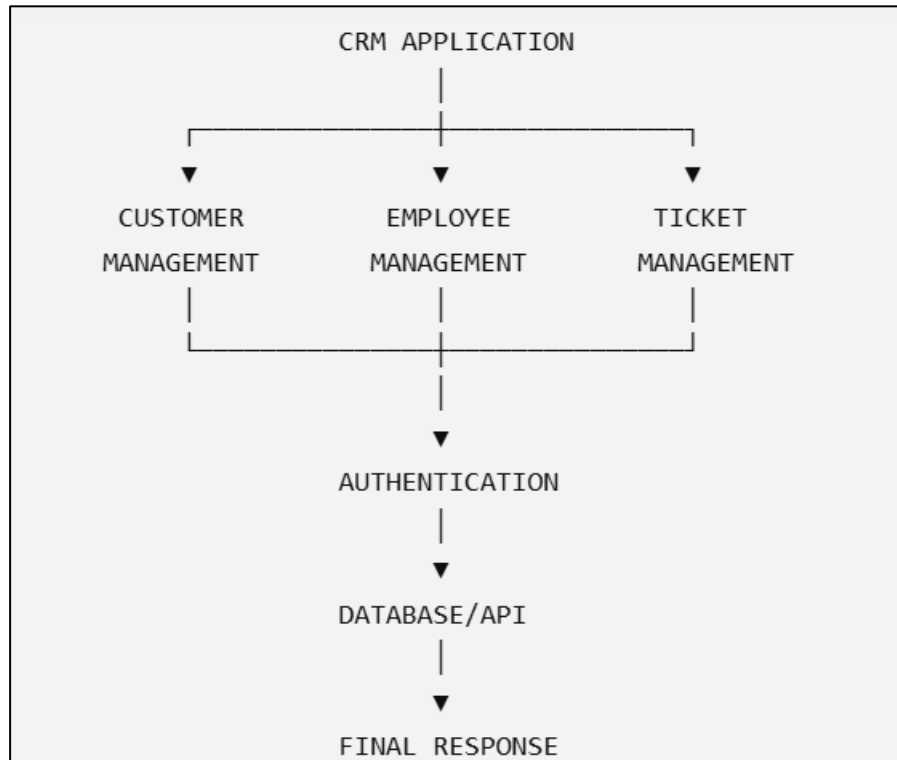


This architecture established the foundation for the subsequent **API Application** and **API Consumption** sections of the project.

Figure 1.13 —Integration with Backend Components

1.13 CRM Application Workflow

The overall CRM workflow developed during the internship can be summarized as:



The implementation of these modules resulted in a functional CRM foundation capable of handling core customer-support management operations.

1.14 Result

The completed CRM application provides a centralized interface for managing the primary entities of the HelpDesk system.

The major functionalities implemented include:

Module	Major Functionality
Customer Management	Add, view, search, update and manage customers
Employee Management	Manage employees, IDs and assigned activities
Ticket Management	Create, manage and associate support tickets
Authentication	Admin login and administrator creation
Validation	Client-side input validation
UI/UX	Dynamic, interactive and modern interface
Database Integration	Persistent storage of application data
API Integration	Communication with backend/external APIs

The CRM application therefore served as the **primary functional layer** of the HelpDesk CRM project and provided the user-facing foundation for the API and database components developed alongside it.

SECTION 2 — API APPLICATION

2.1 Introduction

The API Application was developed as a separate backend component of the HelpDesk CRM project using **ASP.NET Core Web API**. Its purpose was to provide structured HTTP-based communication between the CRM application, database, and external services.

The API layer was developed incrementally according to project requirements. It includes endpoints for API health verification, employee action tracking, employee information retrieval and modification, employee deletion, and employee mobile-number-based API processing. The API application is integrated with the project's MySQL database and also contains API logging functionality for maintaining a record of API operations.

2.2 API Application Architecture

The API application acts as a backend service between the CRM interface and the database/external API.

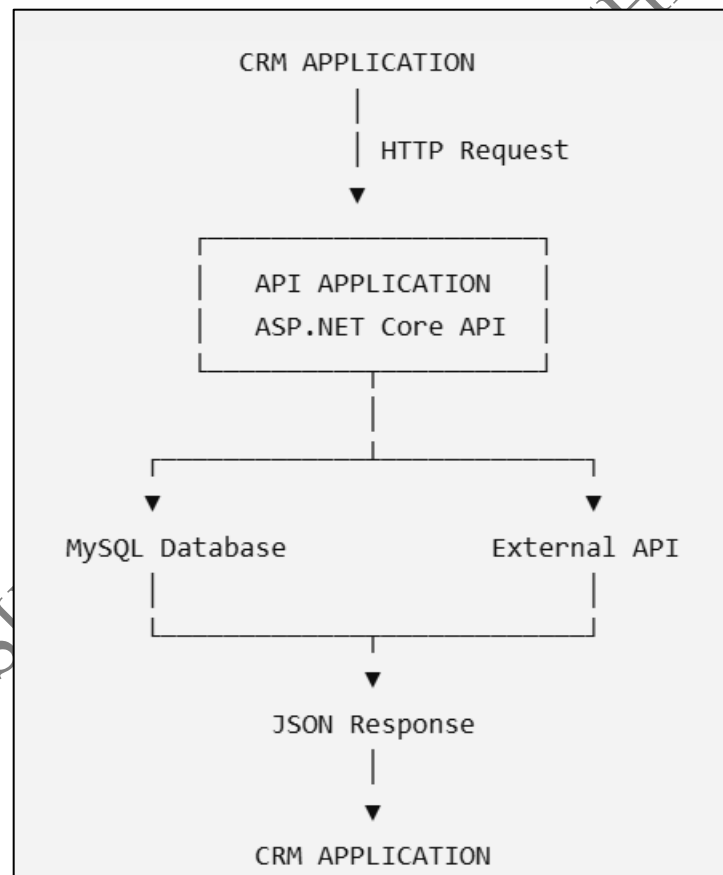


Figure 2.1 — HelpDesk CRM API Architecture

2.3 API Project Structure

The API application was maintained as a separate project named **HelpDeskCRM.API**. The major components include:

- Controllers
- Models
- Data
- Middleware
- Configuration
- Application startup

The API project contains dedicated controllers for testing, employee action attempts, and employee mobile API processing.

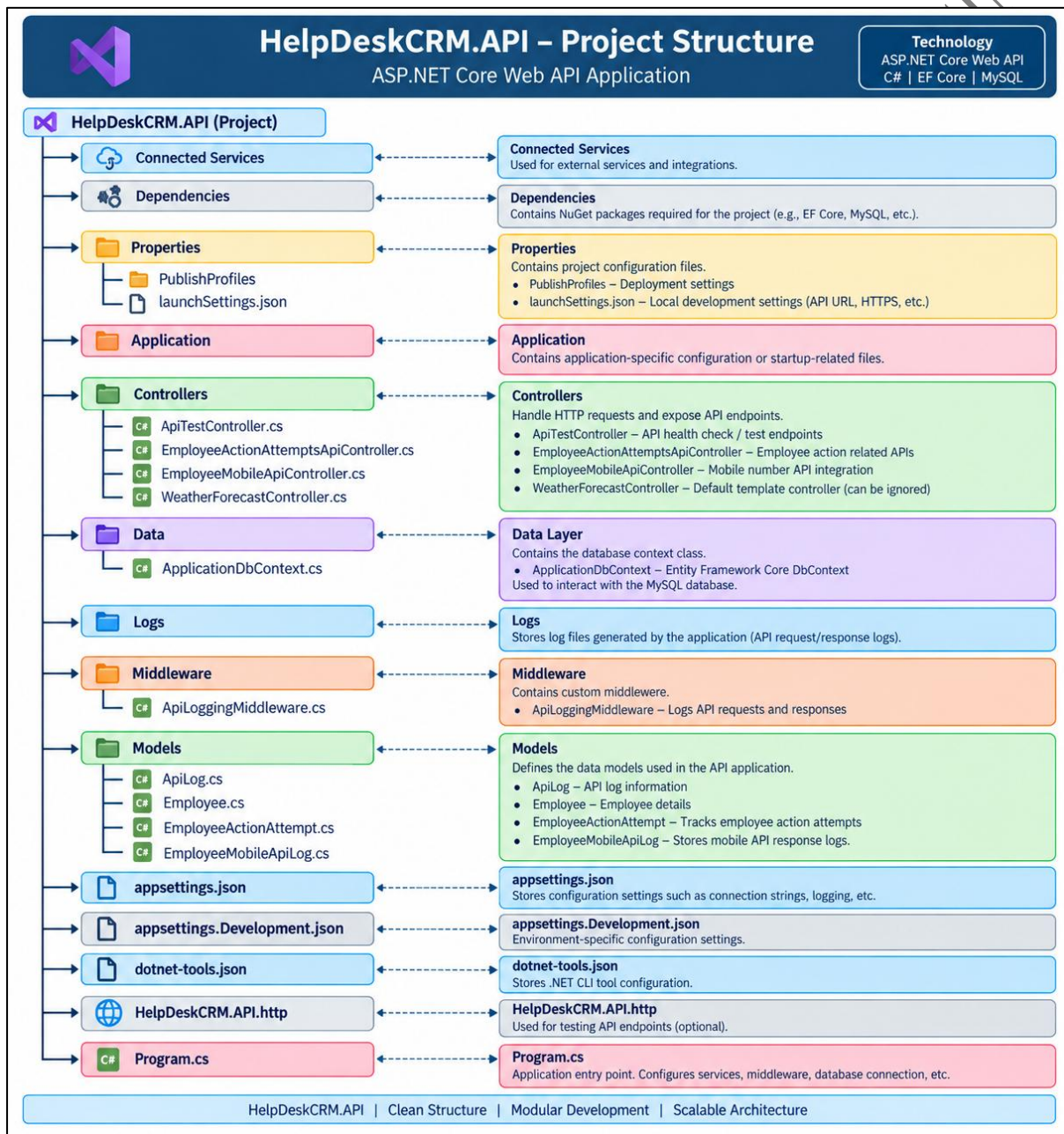


Figure 2.2 — HelpDeskCRM.API Project Structure

2.4 API Response Handling

The API application uses HTTP response codes to communicate the result of operations. The major response categories encountered during development include:

Status Code	Meaning	Project Usage
200	Successful request	Successful GET/API operation
400	Bad Request	Invalid/unsuccessful request
404	Not Found	Requested employee/action record unavailable
403	Forbidden	Authorization/access issue encountered during testing
500	Internal Server Error	Unexpected processing/database error

The response body also provides additional information wherever required, allowing the client to understand the result of the request.

2.5 API Testing Using Postman

Postman was used extensively to test the implemented endpoints independently of the CRM user interface.

For each endpoint, the following were verified:

- HTTP method
- Endpoint URL
- Request parameters
- Request body
- HTTP status code
- Response structure
- Database effect where applicable

The testing process helped verify the API layer before and during integration with the CRM application.

2.6 Open Employee Portal API

The Open Employee Portal endpoint was developed to record an employee's attempt to open the employee portal.

Endpoint : POST

/api/EmployeeActionAttemptsApi/OpenPortal

Input

The API accepts an employee ID through the request body.

Processing

The request is processed through the employee-action tracking mechanism and records the corresponding action.

The response provides information such as:

- Employee ID
- Action name
- Attempt count
- Record ID
- Inserted timestamp
- Updated timestamp
- Status

Response

The endpoint returns the recorded action information when the operation is successful.

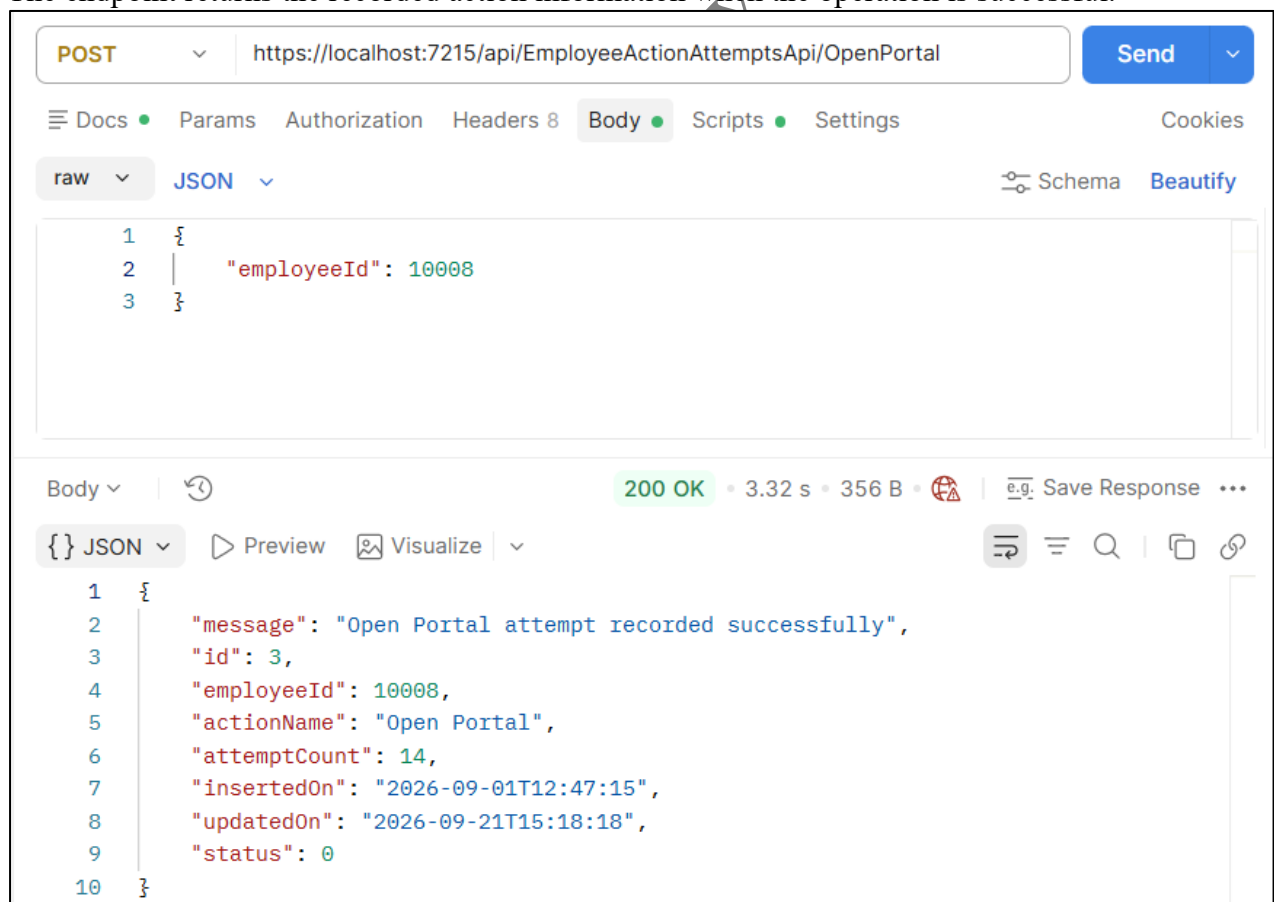


Figure 2.3 — Open Employee Portal API Request & Response

This endpoint forms an important part of the employee action tracking functionality.

2.7 Get All Employee Action Attempts

This endpoint retrieves all employee action-attempt records maintained by the API.

Endpoint : GET

/api/EmployeeActionAttemptsApi/GetAllAttempts

Purpose

The endpoint provides a complete list of recorded employee actions.

The returned information includes:

- Record ID
- Employee ID
- Action name
- Attempt count
- Inserted date/time
- Updated date/time
- Status

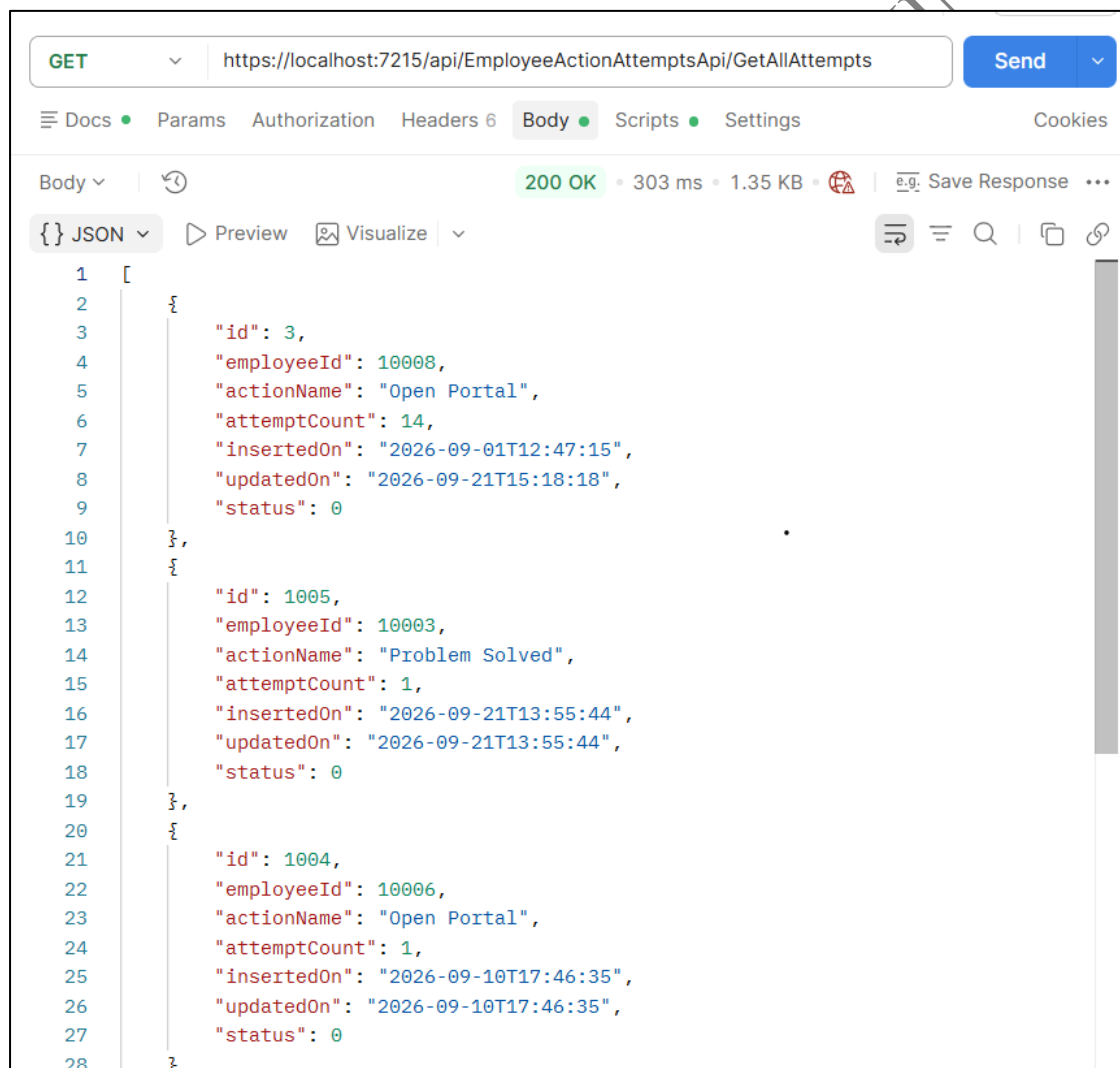


Figure 2.4 — Get All Employee Action Attempts Request & Response

This API provides a centralized way to retrieve employee action-attempt information for monitoring and further processing.

2.8 Get Employee Action Attempts

The API also supports retrieving action attempts for an individual employee.

Endpoint : GET

/api/EmployeeActionAttemptsApi/{employeeId}

Input

The employee ID is passed as a route parameter.

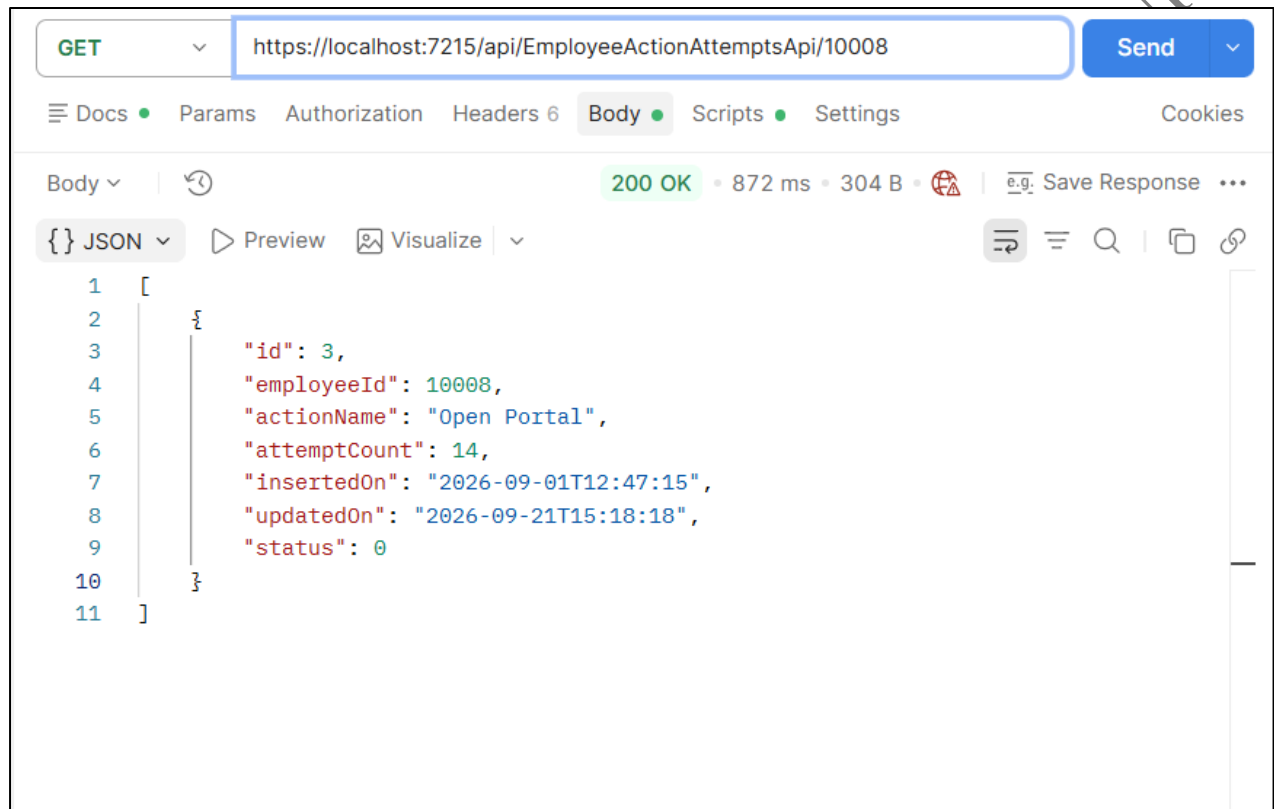


Figure 2.5 — Get Employee Attempts Request & Response

The endpoint retrieves action attempts associated with the specified employee.

If records are available, they are returned as a collection. If no action attempts exist for the specified employee, the API returns a corresponding **Not Found** response.

2.9 Edit Employee API

The Edit Employee API was developed to update employee information through the backend API.

Endpoint : PUT

/api/EmployeeActionAttemptsApi/EditEmployee

Purpose

The endpoint accepts updated employee information and performs the required update operation.

PUT was selected because the operation modifies an existing employee resource.

The API communicates with the database to update the corresponding employee information.

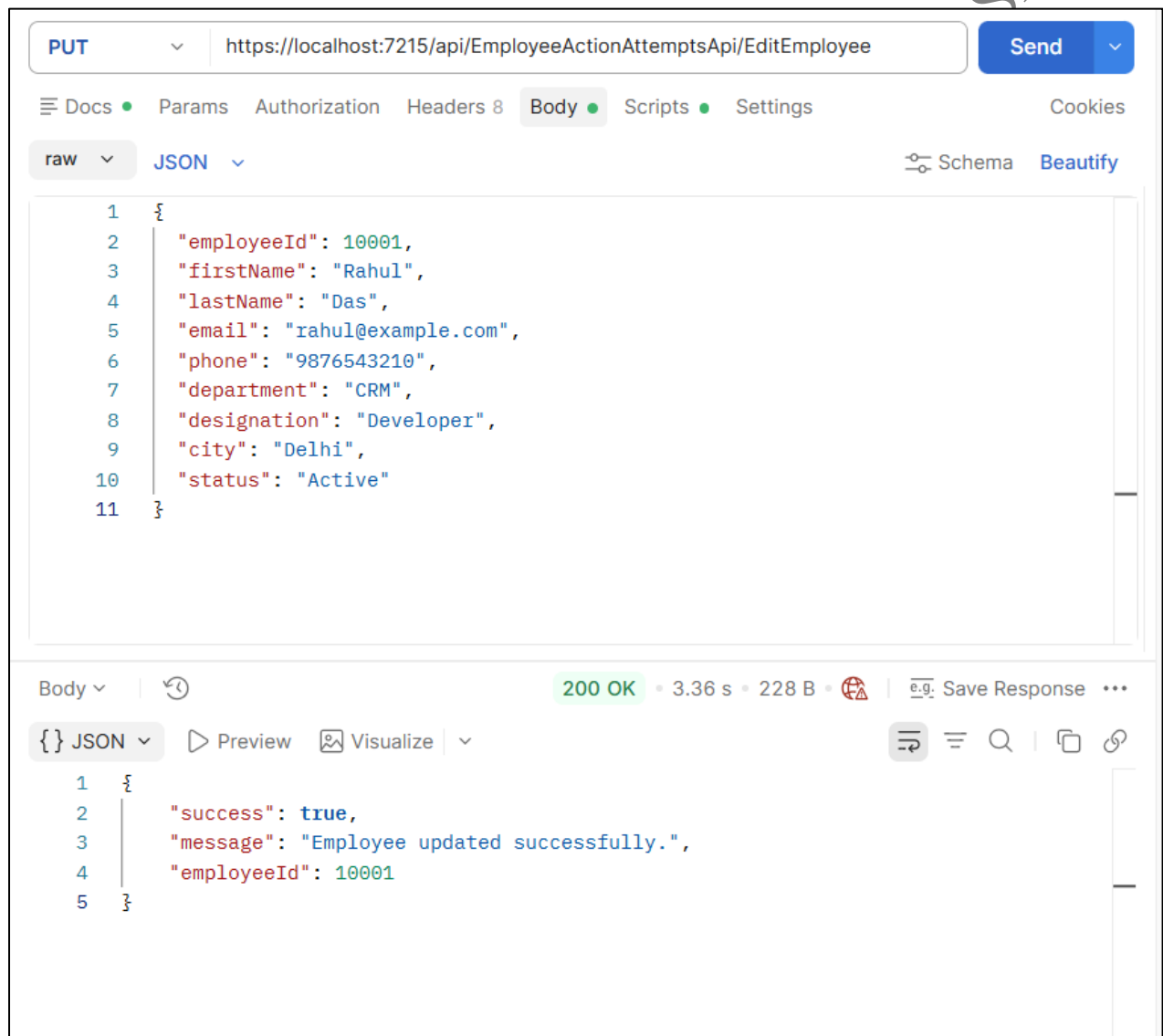


Figure 2.6 — Edit Employee API Request & Response

This endpoint demonstrates the use of the API as an independent backend service for modifying persistent employee information.

2.10 Delete Employee API

The Delete Employee API was implemented to remove an employee record through an HTTP DELETE request.

Endpoint : DELETE

/api/EmployeeActionAttemptsApi/DeleteEmployee/{employeeId}

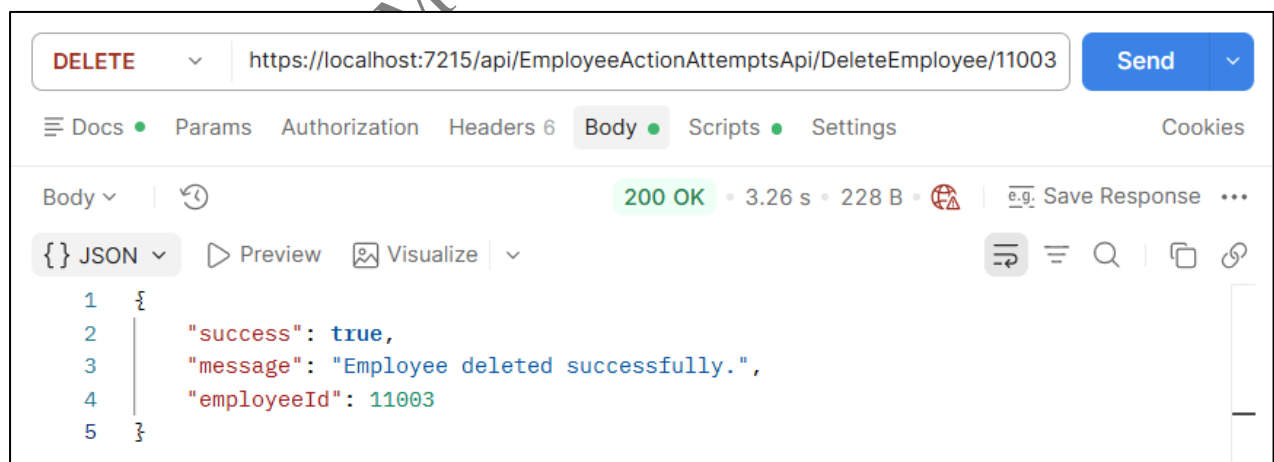
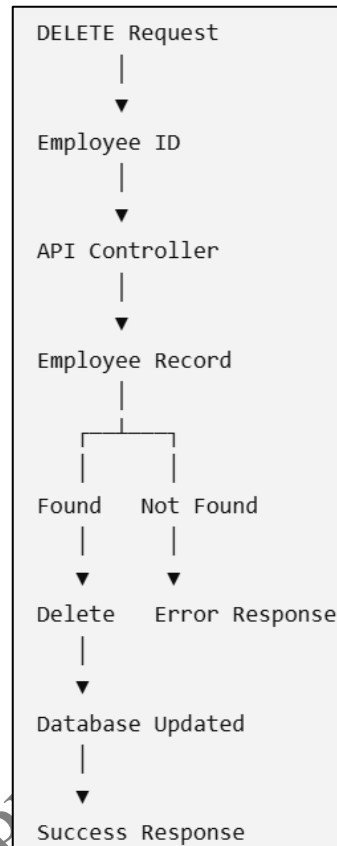


Figure 2.7 — Delete Employee API Request & Response

The successful implementation of this endpoint provided an API-based mechanism for employee deletion and was subsequently tested independently using Postman.

2.11 API Debugging and Troubleshooting

During development and testing, API-related issues were identified and investigated through request testing, response analysis, and application debugging.

One of the notable issues encountered was a **403 Forbidden response** during API communication.

The issue was investigated by reviewing the API request, authorization requirements, endpoint configuration, and response behavior.

The troubleshooting process helped establish a systematic approach to identifying whether an issue originated from:

- Request configuration
- Authorization
- API endpoint
- Application configuration
- Database connectivity
- External API communication

2.12 API Logging

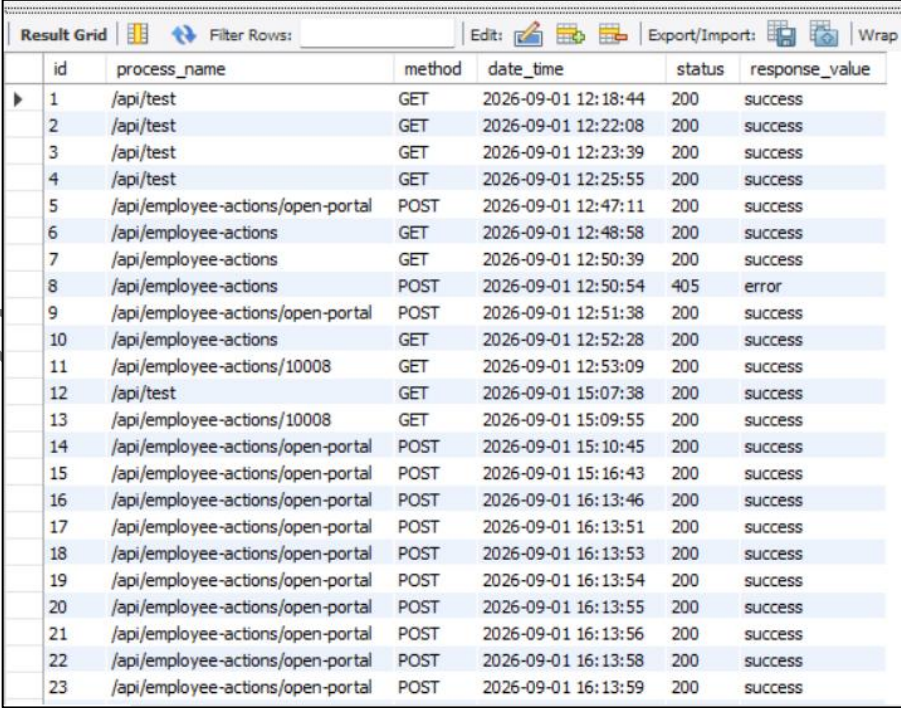
The API application also includes API logging functionality.

API activity can be recorded in the database through the API logging middleware and associated logging model.

The logging system maintains information such as:

- Process name
- HTTP method
- Date/time
- Status
- Response value

This provides a record of API activity and supports monitoring and troubleshooting.

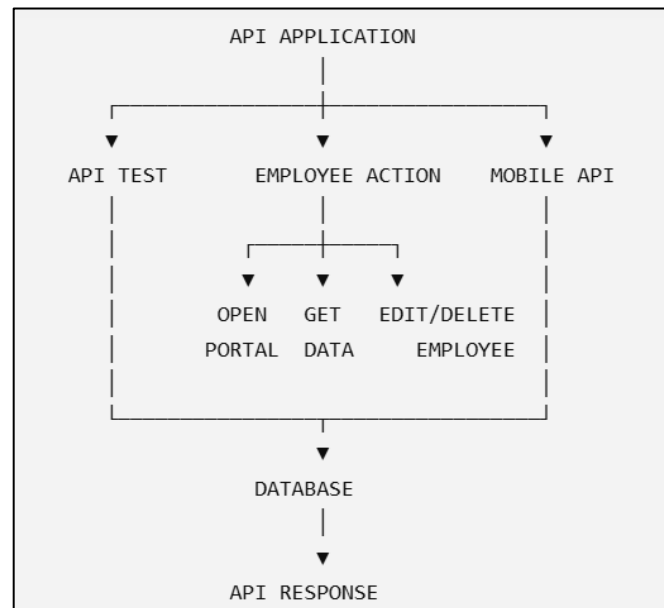


	id	process_name	method	date_time	status	response_value
▶	1	/api/test	GET	2026-09-01 12:18:44	200	success
	2	/api/test	GET	2026-09-01 12:22:08	200	success
	3	/api/test	GET	2026-09-01 12:23:39	200	success
	4	/api/test	GET	2026-09-01 12:25:55	200	success
	5	/api/employee-actions/open-portal	POST	2026-09-01 12:47:11	200	success
	6	/api/employee-actions	GET	2026-09-01 12:48:58	200	success
	7	/api/employee-actions	GET	2026-09-01 12:50:39	200	success
	8	/api/employee-actions	POST	2026-09-01 12:50:54	405	error
	9	/api/employee-actions/open-portal	POST	2026-09-01 12:51:38	200	success
	10	/api/employee-actions	GET	2026-09-01 12:52:28	200	success
	11	/api/employee-actions/10008	GET	2026-09-01 12:53:09	200	success
	12	/api/test	GET	2026-09-01 15:07:38	200	success
	13	/api/employee-actions/10008	GET	2026-09-01 15:09:55	200	success
	14	/api/employee-actions/open-portal	POST	2026-09-01 15:10:45	200	success
	15	/api/employee-actions/open-portal	POST	2026-09-01 15:16:43	200	success
	16	/api/employee-actions/open-portal	POST	2026-09-01 16:13:46	200	success
	17	/api/employee-actions/open-portal	POST	2026-09-01 16:13:51	200	success
	18	/api/employee-actions/open-portal	POST	2026-09-01 16:13:53	200	success
	19	/api/employee-actions/open-portal	POST	2026-09-01 16:13:54	200	success
	20	/api/employee-actions/open-portal	POST	2026-09-01 16:13:55	200	success
	21	/api/employee-actions/open-portal	POST	2026-09-01 16:13:56	200	success
	22	/api/employee-actions/open-portal	POST	2026-09-01 16:13:58	200	success
	23	/api/employee-actions/open-portal	POST	2026-09-01 16:13:59	200	success

Figure 2.8 — API Logs in Database

2.13 API Application Workflow

The complete API workflow implemented in the project can be summarized as:



2.14 Final API Endpoint Summary

This table should ideally appear as the **final major table** of Section 2.

#	Endpoint	Method	Main Function	Status
3	/api/EmployeeActionAttemptsApi/OpenPortal	POST	Record portal-opening attempt	✓
4	/api/EmployeeActionAttemptsApi/GetAllAttempts	GET	Retrieve all action attempts	✓
5	/api/EmployeeActionAttemptsApi/{employeeId}	GET	Retrieve employee-specific attempts	✓
6	/api/EmployeeActionAttemptsApi/EditEmployee	PUT	Update employee	✓
7	/api/EmployeeActionAttemptsApi/DeleteEmployee/{employeeId}	DELETE	Delete employee	✓

2.15 Conclusion

The API Application established a dedicated backend service layer for the HelpDesk CRM project. The implementation included **API health monitoring, employee action tracking, employee retrieval and modification, employee deletion, mobile-based API processing, database interaction, logging, and independent endpoint testing.**

The API endpoints were tested using Postman to verify their request handling, response behavior, and database interaction. The completed API layer provided the foundation for communication between the CRM application, database, and external API services.

SECTION 3 — DATABASE

3.1 Introduction

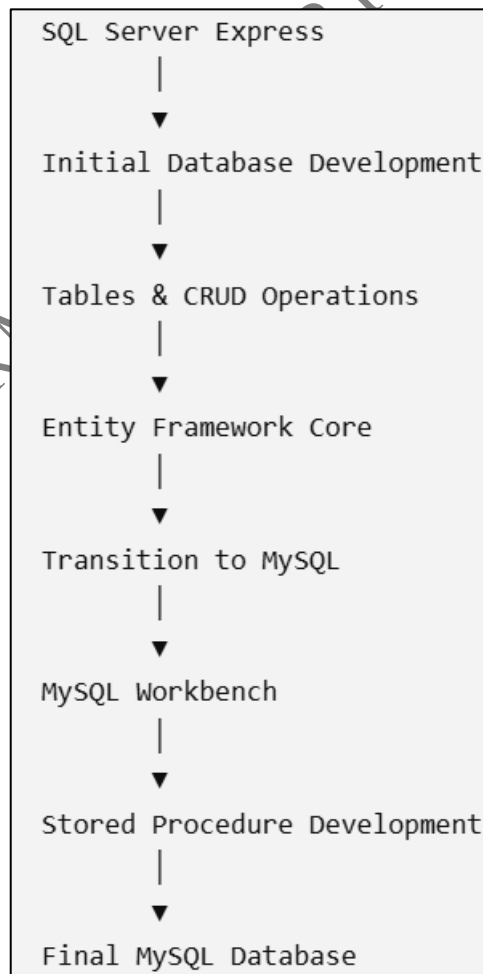
The database layer is a core component of the **HelpDesk CRM Project**, responsible for storing, retrieving, updating, and maintaining application data.

During the internship, database development was carried out using **both SQL Server Express and MySQL**. The project initially used SQL Server Express with **SQL Server Management Studio (SSMS)** for database development and testing. As the project progressed, the database environment was transitioned to **MySQL**, with **MySQL Workbench** used for database management and development.

The **final version of the project is connected to the MySQL database**, which serves as the primary data store for the CRM and API applications.

3.2 Database Development Journey

The database development progressed through the following stages:



Working with both database platforms provided practical experience with database configuration, SQL syntax, application connectivity, and stored-procedure implementation.

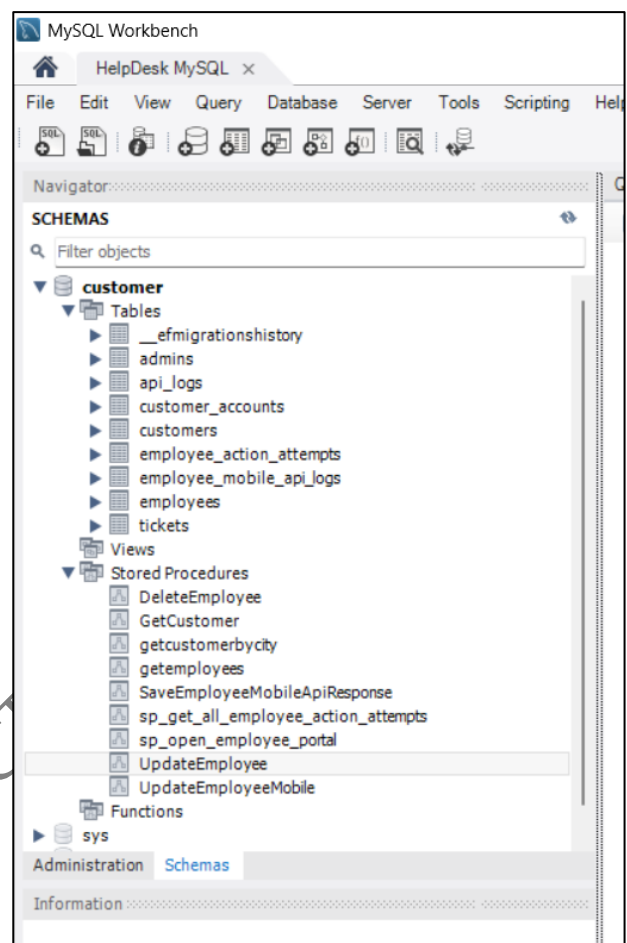
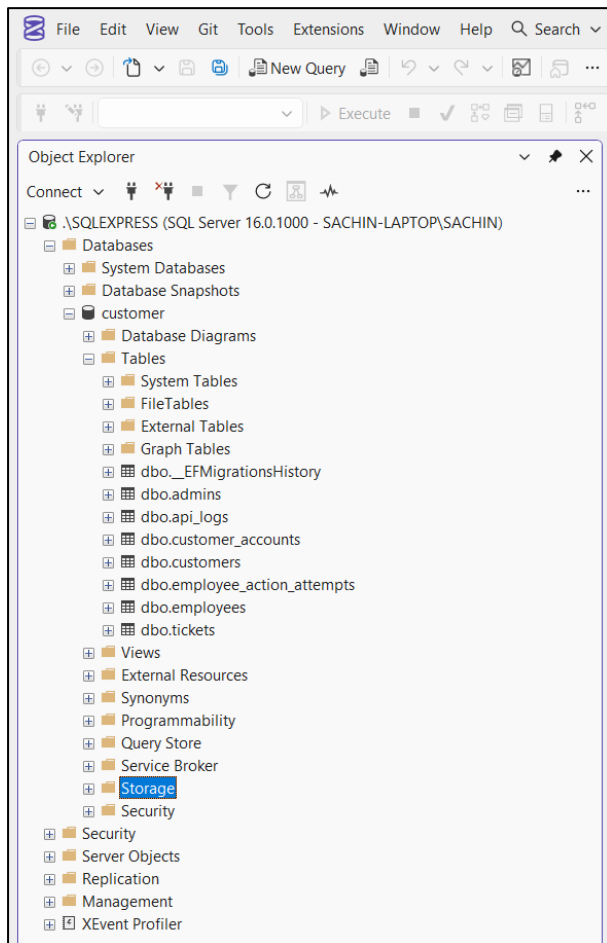


Figure 3.1 Initial SQL Server Express Database in SSMS
Figure 3.2 Final MySQL Database in MySQL Workbench

3.3 SQL Server Express

During the initial development phase, the HelpDesk CRM application was connected to **SQL Server Express** and managed through **SSMS**.

SQL Server Express was used for:

- Initial database creation
- Table creation
- CRUD operations
- Database testing
- Application connectivity
- Entity Framework Core integration

This environment was used during the early stages of application development before the project was transitioned to MySQL.

3.4 Final MySQL Database

The final implementation of the project uses a MySQL database named **customer**. The database was managed using **MySQL Workbench**, where tables, stored procedures, queries, and records were created and verified.

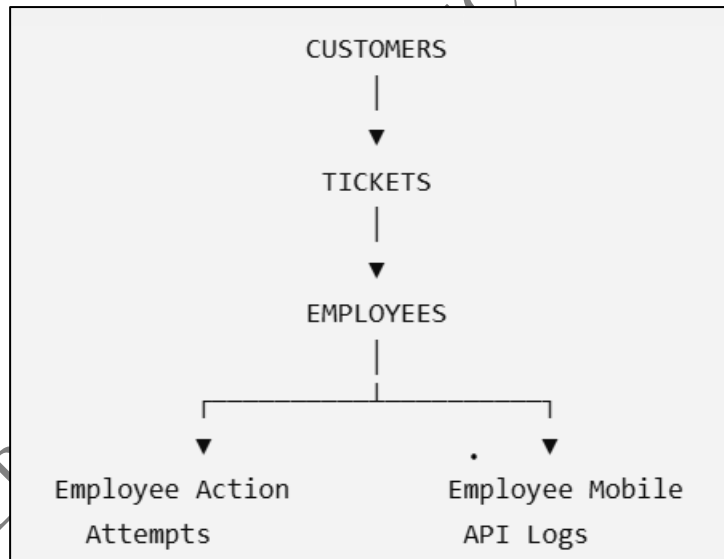
The final database contains the major tables required by the HelpDesk CRM application:

Table	Purpose
admins	Stores administrator information
api_logs	Maintains API activity logs
customer_accounts	Stores customer account information
customers	Stores customer records
employee_action_attempts	Stores employee action attempts
employee_mobile_api_logs	Stores mobile API response information
employees	Stores employee information
tickets	Stores support ticket information
__efmigrationshistory	Maintains Entity Framework migration history

Figure 3.3 — Final customer Database Structure in MySQL Workbench

3.5 Database Structure

The database supports the major modules of the HelpDesk CRM application. The core relationship can be represented as:



Additional tables support:

- Administrator management
- Customer accounts
- API logging
- Entity Framework migrations

This structure allows the CRM application and API application to work with a common persistent data source.

3.6 Entity Framework Core Integration

Entity Framework Core (EF Core) was used to integrate the .NET applications with the relational database.

The `ApplicationDbContext` acts as the central database context and provides access to the application's entities through `DbSet` definitions.

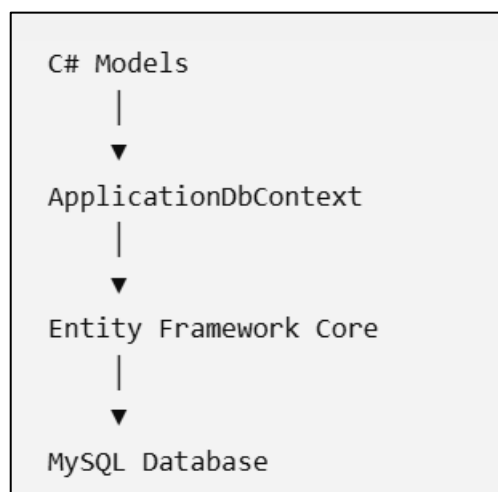


Figure 3.4 — `ApplicationDbContext` in Visual Studio

This integration allowed the CRM and API applications to perform database operations through a structured data-access layer.

3.7 CRUD Operations

CRUD operations were implemented across the application's major entities.

Operation	Purpose
Create	Add new records
Read	Retrieve existing records
Update	Modify existing records
Delete	Remove records

These operations were used for entities such as **customers, employees, and tickets**, while dedicated API endpoints were also developed for employee modification and deletion.

3.8 Stored Procedures

Major Database Development Work

A significant part of the database work involved the **creation, testing, and integration of**

MySQL Stored Procedures.

The final database contains the following stored procedures:

Stored Procedure	Purpose
DeleteEmployee	Handles employee deletion
GetCustomer	Retrieves customer information
getcustomercity	Retrieves customer city-related information
getemployees	Retrieves employee information
SaveEmployeeMobileApiResponse	Saves employee mobile API response
sp_get_all_employee_action_attempts	Retrieves employee action attempts
sp_open_employee_portal	Handles employee portal action
UpdateEmployee	Updates employee information
UpdateEmployeeMobile	Updates employee mobile information

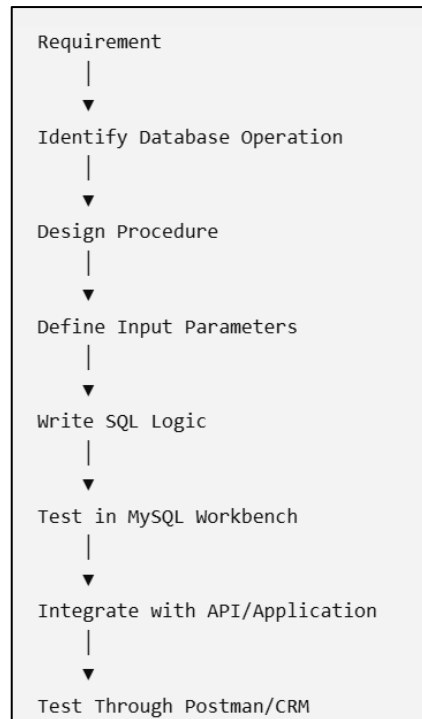
Figure 3.5 — Stored Procedures in MySQL Workbench

This is an important part of the project because the stored procedures were not simply provided as database objects; they were **developed and integrated as part of the application's actual database/API workflow**.

3.9 Stored Procedure Development

The stored procedures were developed to encapsulate specific database operations and make them reusable from the application/API layer.

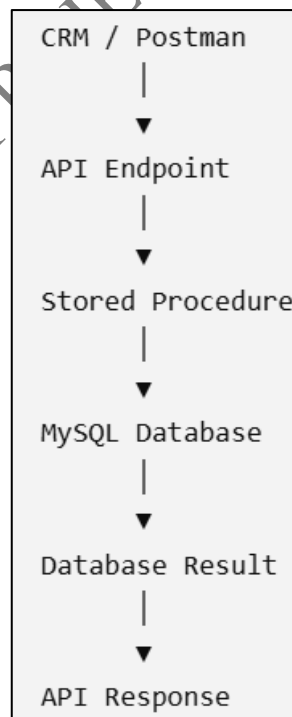
The general workflow was:



3.10 Stored Procedure and API Integration

The stored procedures were integrated into actual API operations.

For example, employee-related operations follow the flow:



Examples include:

- Employee deletion → DeleteEmployee
- Employee update → UpdateEmployee

- Employee mobile update → UpdateEmployeeMobile
- Employee portal action → sp_open_employee_portal
- Employee action retrieval → sp_get_all_employee_action_attempts

This integration demonstrates the connection between the **API Application and Database Layer**.

3.11 API Logging and Response Storage

The database also contains dedicated tables for API-related information.

api_logs

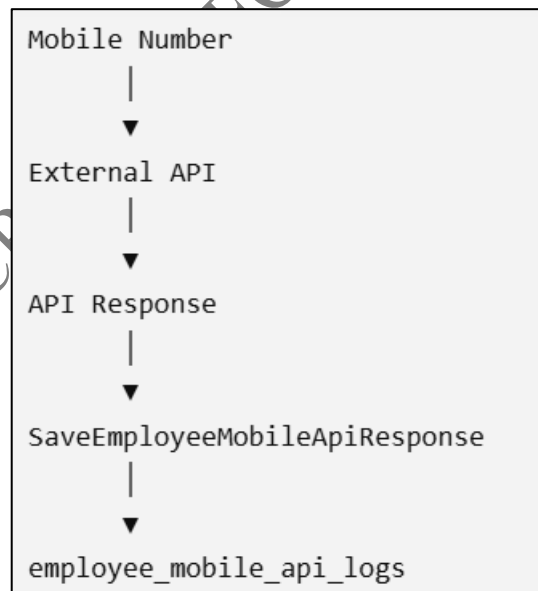
The api_logs table is used for maintaining API activity and related information.

employee_mobile_api_logs

The employee_mobile_api_logs table stores information generated during the employee mobile-number API workflow.

The corresponding stored procedure:

SaveEmployeeMobileApiResponse is used as part of the response-saving workflow.



1 • `SELECT * FROM api_logs`

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap C

id	process_name	method	date_time	status	response_value
8088	/api/EmployeeMobileApi/SaveMobile	POST	2026-09-14 12:18:38	400	error
8089	/api/EmployeeMobileApi/SaveMobile	POST	2026-09-14 12:48:31	400	error
8090	/api/EmployeeMobileApi/SaveMobile	POST	2026-09-14 12:49:12	400	error
8091	/api/EmployeeMobileApi/SaveMobile	POST	2026-09-14 12:53:23	400	error
8092	/api/EmployeeMobileApi/SaveMobile	POST	2026-09-14 12:54:30	400	error
8093	/api/EmployeeMobileApi/SaveMobile	POST	2026-09-14 12:57:07	200	success
8094	/api/EmployeeMobileApi/SaveMobile	POST	2026-09-14 12:58:26	200	success
8095	/api/EmployeeMobileApi/SaveMobile	POST	2026-09-14 14:32:58	200	success
8096	/api/EmployeeMobileApi/SaveMobile	POST	2026-09-14 15:25:00	400	error
8097	/api/EmployeeMobileApi/SaveMobile	POST	2026-09-14 15:28:13	400	error
8098	/api/EmployeeMobileApi/SaveMobile	POST	2026-09-14 15:29:26	200	success
8099	/api/EmployeeMobileApi/SaveMobile	POST	2026-09-14 16:45:20	200	success
8100	/api/EmployeeMobileApi/SaveMobile	POST	2026-09-18 10:12:03	200	success
8101	/api/EmployeeActionAttemptsApi/...	GET	2026-09-21 15:11:22	405	error
8102	/api/EmployeeActionAttemptsApi/...	POST	2026-09-21 15:11:30	500	failure
8103	/api/EmployeeActionAttemptsApi/...	POST	2026-09-21 15:18:18	200	success
8104	/api/EmployeeActionAttemptsApi/...	GET	2026-09-21 15:24:21	200	success
8105	/api/EmployeeActionAttemptsApi/...	GET	2026-09-21 15:32:31	200	success
8106	/api/EmployeeActionAttemptsApi/...	PUT	2026-09-21 15:38:58	500	failure
8107	/api/EmployeeActionAttemptsApi/...	PUT	2026-09-21 15:40:28	200	success
8108	/api/EmployeeActionAttemptsApi/...	DELETE	2026-09-21 15:51:22	500	failure
8109	/api/EmployeeActionAttemptsApi/...	DELETE	2026-09-21 15:51:54	200	success

Figure 3.5 — Mobile API Logs

3.12 SQL Server Express and MySQL Comparison

Aspect	SQL Server Express	MySQL
Management Tool	SSMS	MySQL Workbench
Project Stage	Initial development	Final implementation
ORM	Entity Framework Core	Entity Framework Core
SQL Syntax	T-SQL	MySQL SQL
Stored Procedures	Supported	Developed and used
Final Database	No	Yes

The transition provided practical experience in adapting the application's database layer from one relational database platform to another.

3.13 Final Database Architecture

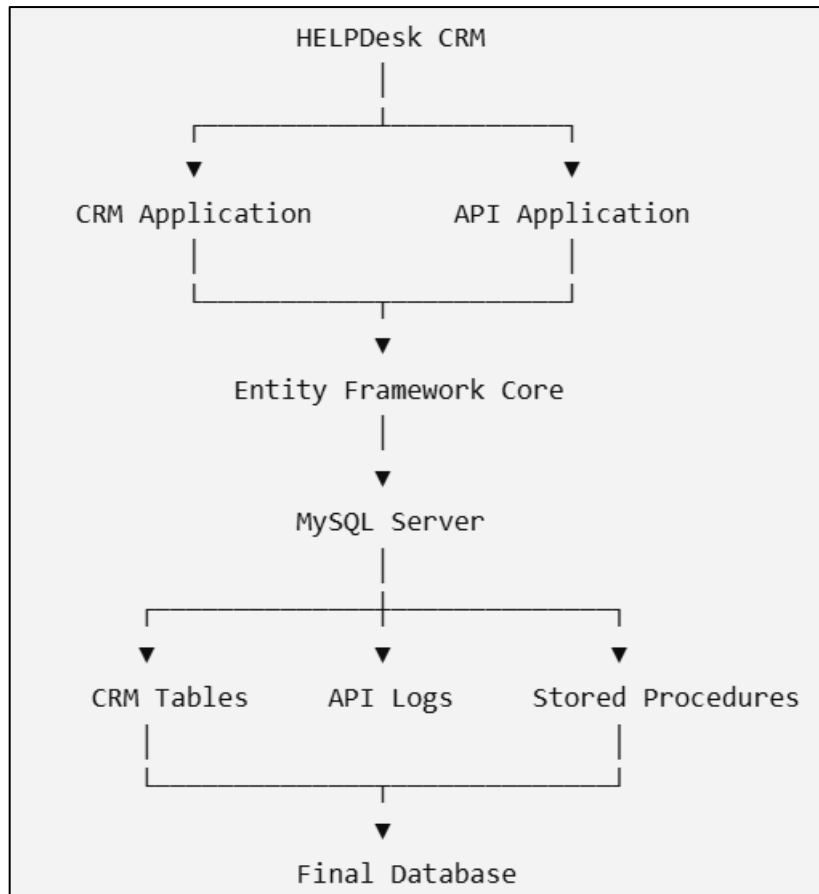


Figure 3.6 — Final Database Architecture

3.14 Outcome

The database development resulted in a fully integrated **MySQL-based database layer** for the HelpDesk CRM project.

The major database work completed during the internship included:

- Initial development using **SQL Server Express and SSMS**
- Transition to **MySQL and MySQL Workbench**
- Database and table design
- CRUD operations
- Entity Framework Core integration
- Development of **9 MySQL Stored Procedures**
- Stored procedure integration with APIs
- API logging
- Employee mobile API response logging
- Database testing and troubleshooting

The database layer therefore provided the persistent foundation connecting the **CRM Application, API Application, and API Consumption** components of the project.

SECTION 4 — API CONSUMPTION

4.1 Introduction

API Consumption was implemented to integrate the **HelpDesk CRM application with an external employee information API** provided during the internship.

The implemented functionality allows the CRM application to send an employee's **mobile number** to the external API and receive the corresponding person information. The returned response is then processed by the application, displayed in the CRM interface, and relevant information is stored in the MySQL database for logging and future reference.

This functionality established an end-to-end integration between the **CRM Application, External API, and MySQL Database**.

4.2 External API

The external API used for the project was:

API: GetPersonNamefromMobileNo

Method: POST

Endpoint: <https://localhost:7215/api/Master/GetPersonNamefromMobileNo>

The API accepts a mobile number in JSON format and returns person-related information corresponding to the supplied number.

Request Content Type

application/json

Authentication

The API requires **Basic Authentication** through the HTTP Authorization header.

For security purposes, the actual authorization credential is not included in this report.

4.3 API Request Structure

The request sent to the external API contains the mobile number in the request body.

Request Body

```
{  
  "MobileNo": "9289815319"  
}
```

The general request structure is:



Figure 4.1 — External API Request in Postman

4.4 Authentication and Request Headers

The external API requires authentication along with the JSON request.
The request contains the following headers:

Header	Purpose
Content-Type	Specifies that the request contains JSON data
Authorization	Provides Basic Authentication for accessing the API

For documentation purposes, the authorization value should be masked:

Authorization: Basic *****

The actual credential should remain confidential and should not be included in the final report.

4.5 API Response

After receiving the request, the external API processes the supplied mobile number and returns a response containing the relevant person information.

The CRM application receives this response and processes the returned data before displaying it to the user.

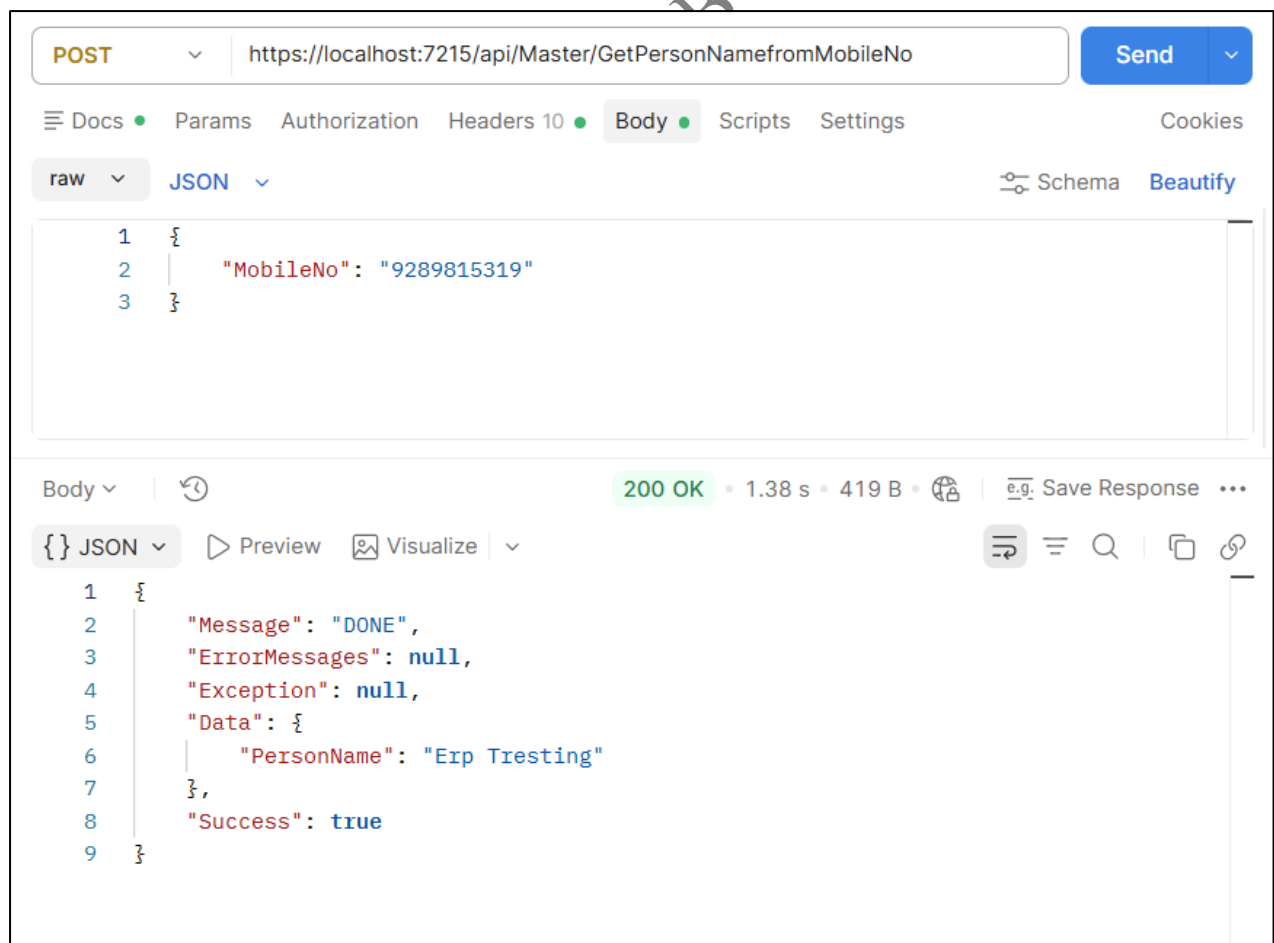


Figure 4.2 — External API JSON Response

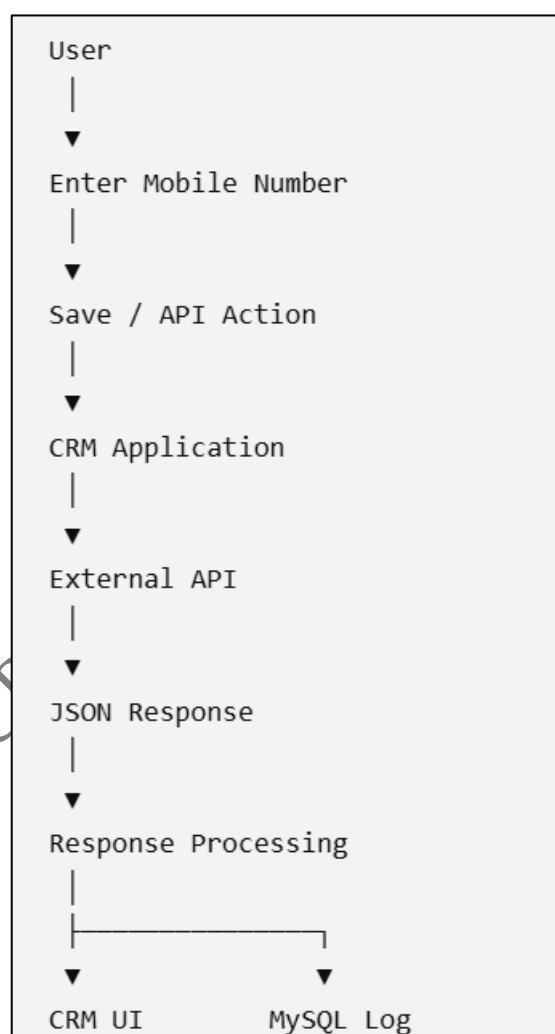
The actual response screenshot should be used in the final report because it provides direct evidence of the successful API integration.

4.6 Integration with CRM Application

The API was integrated into the CRM application through the **Employee Mobile API functionality**.

The user enters a mobile number in the CRM interface and initiates the operation through the appropriate action.

The workflow is:



This allowed the external API response to become part of the CRM workflow rather than remaining an independent API test.

4.7 Displaying API Response Data

The remaining API response information is displayed in the CRM interface in a structured format. The final UI was designed to avoid using one large multiline textbox for all returned information. Individual information fields are presented in a more organized manner wherever applicable.

Consume the Manager API using a mobile number.

Manager API Integration
Enter a mobile number and click **FETCH**. The Manager API response will be displayed below. **FETCH does not save anything to the database.**

Mobile Number *
9289815319
Enter 10 digits starting with 6-9.

FETCH

✓ **API Response**
Exact response received from the Manager API.

Message
DONE

ErrorsMessages
null

Exception
null

Data
{ "PersonName": "Erp Tresting" }

Person Name
Erp Tresting

Success
true

Mobile Number
9289815319

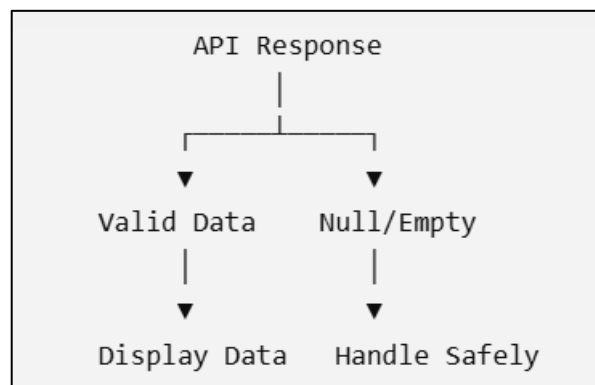
SAVE

Figure 4.3 — External API Data Displayed in CRM

4.8 Handling Null or Empty Responses

The CRM application also handles situations where the external API does not return a valid value for a particular field.

The response-processing workflow can be represented as:



This prevents null or unavailable values from causing unexpected behavior in the CRM interface.

Consume the Manager API using a mobile number.

Manager API Integration

Enter a mobile number and click **FETCH**. The Manager API response will be displayed below. **FETCH does not save anything to the database.**

Mobile Number *

Enter 10 digits starting with 6-9.

FETCH

API Response

Exact response received from the Manager API.

Message

DONE

ErrorMessages

null

Exception

null

Data

null

Person Name

null

Success

true

Mobile Number

9656525633

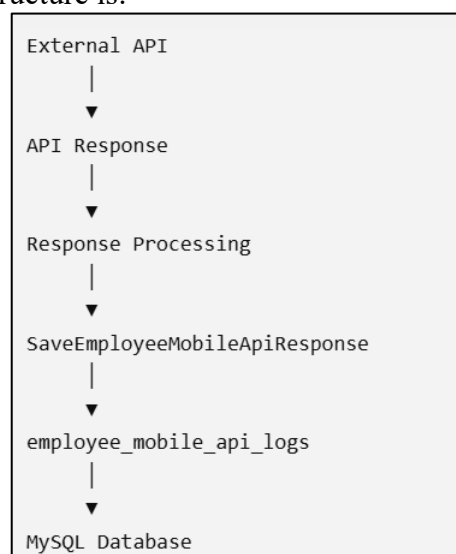
SAVE

Figure 4.4 — Null/Empty Response Handling

4.9 Saving API Response to MySQL

The API consumption functionality was also integrated with the project's MySQL database. After processing the external API response, the relevant information is stored through the **Employee Mobile API logging mechanism**.

The corresponding database structure is:



This creates a persistent record of the API operation.

1 • `SELECT * FROM employee_mobile_api_logs`

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

	id	mobile_no	employee_name	response_message	response_status	created_at
▶	1	9289815319	Erp Tresting	DONE	Success	2026-09-14 11:35:36
	2	9289815319	Erp Tresting	DONE	Success	2026-09-14 12:17:09
	3	9012345678	Dds	DONE	Success	2026-09-14 12:18:04
	4	9856325411	NULL	DONE	Success	2026-09-14 12:18:39
	5	9876501234	NULL	DONE	Success	2026-09-14 12:48:34
	6	9876501234	NULL	DONE	Success	2026-09-14 12:49:13
	7	9876501234	NULL	DONE	Success	2026-09-14 12:53:26
	8	8865231545	NULL	DONE	Success	2026-09-14 12:54:31
	9	9289815319	Erp Tresting	DONE	Success	2026-09-14 12:57:09
	10	9289815319	Erp Tresting	DONE	Success	2026-09-14 12:58:27
	11	9876543210	Test	DONE	Success	2026-09-14 14:32:59
	12	9876123450	NULL	DONE	Success	2026-09-14 15:25:00
	13	9876123450	NULL	DONE	Success	2026-09-14 15:28:15
	14	9876543210	Test	DONE	Success	2026-09-14 15:29:27
	15	9123456789	Agarwal	DONE	Success	2026-09-14 16:45:23
	16	9289815319	Erp Tresting	DONE	Success	2026-09-18 10:12:04
	17	9289815319	Erp Tresting	DONE	Success	2026-09-18 11:19:25
	18	9289815319	Erp Tresting	DONE	Success	2026-09-18 12:19:38
	19	9289815319	Erp Tresting	DONE	Success	2026-09-18 12:57:39
	20	9289815319	Erp Tresting	DONE	Success	2026-09-21 09:46:00
	21	9595656565	NULL	DONE	Success	2026-09-21 10:34:26
•	NULL	NULL	NULL	NULL	NULL	NULL

Figure 4.5 — Employee Mobile API Logs in MySQL

4.10 End-to-End API Consumption Workflow

The complete implementation can be represented as:

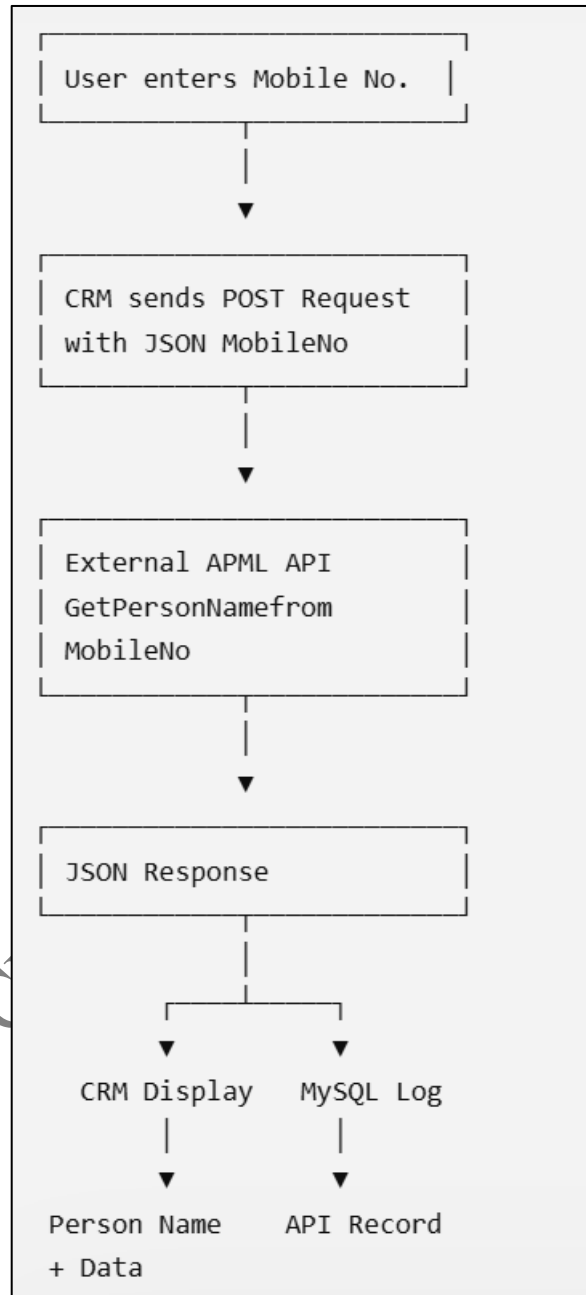


Figure 4.6 — Complete API Consumption Workflow

4.11 API Testing

The external API was tested independently using **Postman** before and during integration with the CRM application.

The testing verified:

- Correct HTTP method
- Correct API endpoint
- JSON request body
- Required authentication
- API response
- Response processing
- CRM display
- Database logging

The API request used for testing follows the structure:

POST :

<https://localhost:7215/api/Master/GetPersonNamefromMobileNo>

with:

```
{  
  "MobileNo": "9289815319"  
}
```

4.12 API Consumption and Database Integration

The complete feature connects three major components of the project:

Component	Responsibility
CRM Application	Accepts mobile number and displays result
External API	Provides person information
MySQL Database	Stores relevant API response/log information

This can be summarized as:

Mobile Number → External API → JSON Response → CRM Display → MySQL Storage

4.13 Final Outcome

The API Consumption implementation successfully integrated an external employee information service into the HelpDesk CRM application.

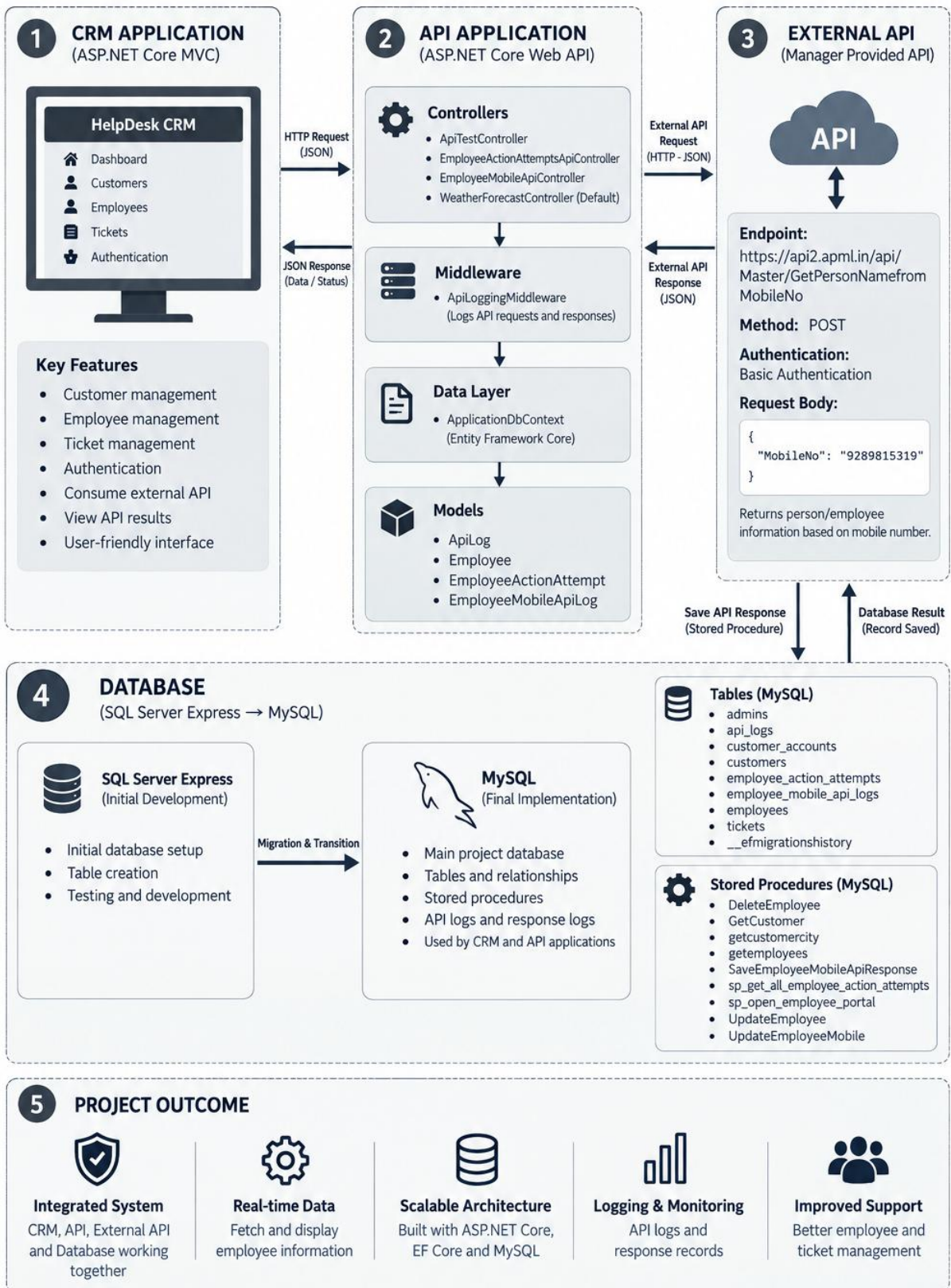
The final workflow allows the system to:

- Accept a mobile number from the CRM interface.
- Send a POST request to the external API.
- Authenticate the request.
- Receive and process the JSON response.
- Display the **Person Name** separately in the CRM.
- Display the remaining returned information in a structured format.
- Handle null or empty response values.
- Store relevant API information in the MySQL database.

This completed the integration between the **CRM Application, External API, API layer, and MySQL database**, forming an important part of the overall HelpDesk CRM workflow.

HelpDesk CRM – Complete Project Architecture

CRM Application | API Application | External API Integration | Database (SQL Server & MySQL)



HelpDesk CRM – From Support to Better Service

CONCLUSION

The project involved the development of a complete **CRM Application using ASP.NET Core MVC**, covering **Customer Management, Employee Management, Ticket Management, and Authentication**. Alongside the CRM application, a dedicated **ASP.NET Core Web API** was developed to support backend operations and communication with the application.

The database component involved practical experience with both **SQL Server Express and MySQL**. The project was initially developed using SQL Server Express and was later transitioned to **MySQL**, which became the final database environment. **Entity Framework Core, CRUD operations, database logging, and MySQL Stored Procedures** were implemented as part of the project.

Another significant component was the integration of an **external API** for retrieving person information using a mobile number. The API response was processed, displayed within the CRM interface, and relevant information was stored in the database. This established an integrated workflow between the **CRM Application, API Application, External API, and MySQL Database**.

Throughout the internship, the project also involved practical **testing, debugging, API verification, database troubleshooting, UI improvements, and integration of different software components**. Working through these stages provided a better understanding of how individual technologies combine to form a complete software solution.

Overall, the internship was a valuable learning experience that strengthened my understanding of **.NET development, web applications, APIs, databases, and real-world software development practices**. The completed HelpDesk CRM Project represents the practical application of the knowledge and skills acquired during the internship and provides a foundation for further development and enhancement.

Sachin Kumar Sinha